



Oracle 白皮书
2013 年 6 月

Oracle Multitenant

执行概要.....	1
本文组织结构	3
对于不需要了解技术细节的读者	3
对于希望充分了解技术细节的读者	3
章节概述	3
Oracle Multitenant 解决的客户难题	5
努力实现最大的整合密度	5
标准化降低运营支出	5
标准化提供更多降低运营支出和资本支出的机会	5
供应数据库	7
修补和升级 Oracle 数据库软件版本	7
Oracle Multitenant 高级描述	9
多租户架构的静态方面：水平分区的数据字典和可插拔性	12
表：最终的逻辑实体	12
非 CDB 架构中的单体数据字典	12
多租户架构的水平分区数据字典	14
方法概括	14
PDB 和 <i>root</i> 的实际定义	15
方法的内部概况	16
针对 PDB 实体的操作：拔出/插入、克隆、创建、删除	17
机器间的拔出/插入操作	17
跨不同操作系统、芯片组或字节序的拔出/插入	18
拔出/插入以修补 Oracle 版本	19
拔出/插入以响应 SLA 变化	22
克隆 PDB	22
使用完整副本克隆 PDB	23
使用快照复制克隆 PDB	23
从拔出的 PDB 克隆	24
使用 GoldenGate 复制作为拔出/插入的替代方法来进行远程克隆	24
<i>clone PDB</i> SQL 语句的语法	25
创建 PDB	25
<i>create PDB</i> SQL 语句的语法	26
删除 PDB	26
<i>drop PDB</i> SQL 语句的语法	26
为什么采用 SQL 语句实现 <i>create PDB</i> 、 <i>clone PDB</i> 、 <i>drop PDB</i> 以及拔出/插入操作	
十分重要？	27
针对 Data Guard 保护的 CDB 执行的拔出/插入和 <i>clone PDB</i> 操作	27
如何将非 CDB 用作 PDB	29
直接将 12.1 非 CDB 用作 PDB	29
采用非 CDB 的内容	29

多租户架构的动态方面：Oracle 实例、用户和会话.....	31
用户、角色和授予的通用性	31
本地用户和本地角色	31
通用用户和通用角色	31
特权和通用角色的通用授予	32
客户创建的通用用户和通用角色	32
服务和会话	33
为已建立的会话更改当前容器	33
SGA 为逻辑虚拟化	34
数据字典视图和性能视图	36
每个 CDB 的选择 和 每个 PDB 的选择.....	39
只能对 CDB 整体进行的选择	39
Oracle 数据库软件版本和平台类别	39
Spfile、控制文件和密码文件	39
Data Guard、RMAN 备份、重做和撤销	39
字符集.....	40
CDB 级初始化参数和数据库属性	41
AWR 报告	41
可对各个 PDB 分别进行的选择	41
PDB 时间点恢复	41
PDB 的 <i>即席</i> RMAN 备份	42
alter system flush Shared_Pool	42
PDB 可设置的初始化参数和数据库属性.....	42
ORA-65040 错误	42
CDB 内 PDB 间的资源管理	43
在 12.1 中由 CDB 级计划控制的计算资源	43
份额和上限模型	44
12.1 中 CDB 级计划管理会话、CPU、Oracle 并行服务器进程以及文件 i/o 的方式	44
PDB 和 RAC 实例的关联	45
CDB 中单独的 PDB 和非 CDB	47
总结.....	48
附录 A：Oracle 数据库文档库中有关多租户架构的介绍	49

执行概要

Oracle Multitenant 是 Oracle Database 12c 企业版的一个新选件，它通过简化整合、供应以及升级等过程来帮助客户降低 IT 成本。它由一个新架构提供支持，该架构允许在一个容器数据库中容纳多个可插拔数据库。而且它充分补充完善了其他选件，其中包括 Oracle Real Application Clusters 和 Oracle Active Data Guard。现有的数据库无需任何更改即可用作可插拔数据库；而且应用程序的其他层也不需要任何更改。Oracle Multitenant 的优势得自于它实现了一种纯粹的部署选择。下面列表例举了其最具吸引力的优势。

- **整合密度高。**单个容器数据库中的多个可插拔数据库共享容器数据库的内存和后台进程，从而相对使用旧架构的单个数据库而言，您能够在特定平台上运行更多可插拔数据库。基于模式的整合能够提供同样的优势，但是采用基于模式的整合存在比较大的障碍，而且会导致持续的运营问题。而新架构则扫清了这些采用障碍并消除了运营问题。
- **通过 SQL 实现快速供应和克隆。**一个可插拔数据库可从一个容器数据库中拔出并插入到另一个容器数据库中。或者，您可以在同一个容器数据库中克隆一个可插拔数据库，或从一个容器数据库克隆到另一个容器数据库中。这些操作，以及创建可插拔数据库的操作，均可使用新的 SQL 命令在数秒内完成。如果底层文件系统支持精简供应，那么通过在 SQL 命令中使用关键字 *snapshot* 几乎瞬间即可克隆数 TB 的数据。
- **快速修补和升级的新范式。**投入时间和精力修补一个容器数据库，就等于修补其包含的所有可插拔数据库。要修补单个可插拔数据库，只需将其拔出，再插入到具有不同 Oracle 数据库软件版本的另一个容器数据库中。
- **将多个数据库作为一个整体来管理。**通过将现有数据库作为可插拔数据库进行整合，管理员可以将多个数据库作为一个整体来管理。例如，诸如备份和灾难恢复等任务可以在容器数据库级别执行。
- **可插拔数据库之间的动态资源管理。**Oracle Database 12c Resource Manager 经过了扩展，它使用特别的功能来即时控制容器数据库中可插拔数据库之间的资源争用。

本文介绍新的 Oracle 数据库架构及其带来的新功能以及随之而来的优势。

免责声明

以下内容旨在概述产品的总体发展方向。该内容仅供参考，不可纳入任何合同。其内容不构成提供任何材料、代码或功能的承诺，并且不应该作为制定购买决策的依据。所描述的有关 Oracle 产品的任何特性或功能的开发、发布和时间安排均由 Oracle 自行决定。

本文组织结构

对于不需要了解技术细节的读者

您可以只阅读这些章节：

- “*Oracle Multitenant 解决的客户难题*”，第 5 页
- “*Oracle Multitenant 高级描述*”，第 9 页
- “*总结*”，第 48 页

还可以略读以下章节，这将颇有助益：

- “*如何将非 CDB 用作 PDB*”，第 29 页
- “*每个 CDB 的选择和 每个 PDB 的选择*”，第 39 页
- “*CDB 内 PDB 间的资源管理*”，第 43 页
- “*CDB 中单独的 PDB 和非 CDB*”，第 47 页

对于希望充分了解技术细节的读者

您当然应该从头到尾仔细阅读全文。

章节概述

第 5 页上的“*Oracle Multitenant 解决的客户难题*”描述了以下三个问题：力争将很多数据库整合到一个平台上，以获得最大的投资回报；供应数据库；以及为很多数据库进行 Oracle 版本修补。

之后第 9 页上的“*Oracle Multitenant 高级描述*”将介绍描述新架构特征的术语，特别是多租户容器数据库（缩写为 CDB）和可插拔数据库（缩写为 PDB），并概要介绍（而不是详细说明）高级功能及其如何解决客户难题。

之后在第 12 页上的“*多租户架构的静态方面：水平分区的数据字典和可插拔性*”中，我们将描述客户在 Oracle 数据库 12.1¹ 之前的架构中面临的难题以及 12.1 带来的新架构如何消除问题根源。简而言之，该新架构提供数据库内的虚拟化，这样“超级数据库”，即容器数据库可以包含“子数据库”，即可插拔数据库。我们将会看到对客户系统使用“可插拔数据库”这一术语的原因。

接下来在第 17 页上的“*针对 PDB 实体的操作：拔出/插入、克隆、创建、删除*”中，我们将了解到，在多租户架构环境中通过 SQL 语句实现的这些操作如何为供应和修补 Oracle 版本带来了新的范式。

1. 我们将使用 12.1 作为 Oracle Database 12c 的简称，使用 11.2 作为 Oracle Database 11g 的简称，依此类推。

我们将在[第 29 页](#)上的“*如何将非 CDB 用作 PDB*”中简要介绍如何将一个 12.1 之前的数据库用作一个 PDB。

之后在[第 31 页](#)上的“*多租户架构的动态方面：Oracle 实例、用户和会话*”中，我们将了解为何现在需要区分通用用户和本地用户以及如何创建会话 — 特别是创建一个从不会脱离其连接的 PCB 的会话。我们还将了解如何通过使用所属系统的标识符标记每个结构来实现 SGA、重做日志以及 undo 表空间的*逻辑*虚拟化，而不是*物理*虚拟化。

在您对新架构有所了解后，我们将在[第 39 页](#)上的“*每个 CDB 的选择和 每个 PDB 的选择*”中介绍：PDB 保留了 12.1 之前数据库享受的何种程度的自由度，以及为了实现整合目标，不可避免地需要牺牲哪些方面。

接下来在[第 43 页](#)上的“*CDB 内 PDB 间的资源管理*”中，将描述一个 CDB 中各 PDB 之间的资源管理。

在[第 47 页](#)上的“*CDB 中单独的 PDB 和非 CDB*”中，针对托管一个应用后端这个目标，我们将对 CDB 中专用作此用途的单个 PDB 和 12.1 之前架构的数据库进行比较。我们将看到单独的 PDB 毫不逊色，颇有优势。

在[第 48 页](#)上的“*总结*”中，我们将对本文进行概括总结。后面还有一个附录：

- “*附录 A：Oracle 数据库文档库中有关多租户架构的介绍*”，[第 49 页](#)。

Oracle Multitenant 解决的客户难题

成百上千的数据库分散运行于客户站点中几乎同样多的机器上，导致站点规模庞大，这种情况如今是相当常见。随之而来的不菲开支促使组织主动采取行动，将众多数据库集中到一起，换言之就是整合，以便降低成本。过去数十年中，数据库管理员花费了比其预想更多的时间，用于供应数据库，为众多数据库分别进行 Oracle 版本修补，规划、建立以及管理备份和灾难恢复方案，或者分别管理每个数据库（作为每个数据库各自的持续活动）。

努力实现最大的整合密度

标准化本质上是有益的 — 这非常简单，很容易理解。通过标准化，能够进一步允许将符合某特定标准的多个数据库整合到实施该标准的单个平台上。

标准化降低运营支出

一个组织如果拥有很多数据库，会导致很高的拥有成本，而这些数据库之间的差异导致此情况更加严重。最明显的差异因素是硬件和操作系统的类型、选件的选择以及 Oracle 数据库软件版本的选择 — 体现在最细粒度的一次性补丁上。其他差异，如执行计划备份方案的方式以及实现灾难恢复的方式，都会对运营支出产生影响。

很多客户已经发现，控制复杂性的第一步是，将尽可能多的数据库托管到尽可能少的硬件平台上。操作系统虚拟化乍看起来似乎是一个具有吸引力的权宜之计，通过该方法，各个数据库就好象原先各自运行在自己的平台上一样可以实现自主管理。但是，这实际是通过操作系统虚拟化尽量减少数据库间允许存在的差异，来实现管理成本的节省。因此，将很多数据库整合到少量机器上的客户已认识到操作系统虚拟化用以解决的问题是他们所没有的问题 — 因此，他们更愿意使用原生的没有虚拟化的操作系统，部署尽可能少的 Oracle 主目录安装，换言之，他们对硬件类型、操作系统版本和补丁级别，特别是他们支持的 Oracle 数据库的特定配置进行标准化。

标准化提供更多降低运营支出和资本支出的机会

一直在努力追求整合的客户如今已经认识到，由于以下事实，整合密度受到限制：每增加一个数据库都会带来明显增加的内存和 CPU 需求，这是因为每个数据库都有自己的 SGA 和后台进程组 — 如果部署的是 Oracle Real Application Clusters，这些自然会成倍增加。至此他们认识到，重要的指标不是平台可以支持的数据库的密度，而是平台可以支持的应用后端的密度。

我们定义的术语“应用后端”指的是在数据库内为特定应用程序实现持久性机制的构件组。对于每个应用程序，必然存在一种用来安装应用后端的机械性方案。该方案可能完全由 SQL*Plus 脚本组成，可能由脚本和数据泵导入文件及命令的组合组成，也可能由发出相应 SQL 语句的特定用途客户端程序组成。

不过该机械性方案确实会存在。我们使用术语“*构件*”表示应用后端的安装方案对数据库所做的任何更改的结果。最明显的示例是 *DBA_Objects* 中列出的对象，如表、索引、视图、PL/SQL 包等等。但是某些构件，如用户、角色和表空间不是此种意义上的对象。同样，向用户或角色授予特权或角色得到的结果也不是此种意义上的对象。还有应用程序表中作为应用程序元数据的数据也是如此。

传统上，每个应用后端都被安置在自己的专用数据库中。但是，通过依次运行每个应用后端的安装方案，也可以将许多不同的应用后端安装到同一个数据库中。此方法通常称为*基于模式的整合*，我们将在本文中使用该术语²。客户表示，如果按每个平台支持的应用后端数来衡量，相对每个应用后端都在自己数据库中的情况（以下称为*专用数据库模式*），基于模式的整合支持明显更大的整合密度。整合密度取决于应用后端的属性，如它需要的数据库内存大小以及它必须支持的吞吐量。此外，大量减少数据库的数量，会带来显著的管理成本节省。

不过，基于模式的整合也存在一些众所周知的缺点。

- *名称冲突可能会阻碍基于模式的整合*。一些应用后端完全在一个模式中实施。这种情况下，通常可以自由选择模式名称，只需在中央客户端配置文件中指定即可。这样的应用后端通常可以轻易地安装在同一个数据库中（但是尽管如此，还是有一些潜在问题，在后面将会看到）。当然，正是这种用例，才产生“基于模式的整合”这一术语。

其他应用后端在多个模式中实施，这意味着存在跨模式的引用，由此也会产生对设计采用的特定模式名称的依赖。当然，有可能不同的应用后端中使用相同的模式名称。但是一个模式的名称在整个数据库中必须是唯一的，因此那些具有冲突模式名称的应用后端无法安装在同一个数据库中，除非至少其中一个应用后端更改模式名称。这种更改通常被认为代价过高。

此外，其他对象的名称在整个数据库中也是唯一的。如角色、目录、版本、公共同义词、公共数据库链接和表空间。即使在每个应用后端在单一模式中实施的情况下，这些其他对象的名称也可能会冲突，从而阻碍这些应用后端安装在同一个数据库中。

- *基于模式的整合安全性较弱*。安装特定应用后端的人员需要作为拥有 *Create User*、*Alter User* 和 *Drop User* 等强大特权的数据库用户来进行身份验证。之后该人员可以对同一数据库中的任何其他应用后端进行更改，因而可以阅读、更改其机密数据。此外，其实施使用多个模式的设计不良的应用后端，可能会依赖系统特权，如 *Select Any Table*、*Delete Any Table* 等。这种情况下，我们举例来说，SQL 注入漏洞可能会允许一个应用后端的运行时用户读取或更改另一个应用后端的数据。

2. 稍后我们将会看到基于模式的整合是一种*数据库内整合*，我们将把它与多租户架构带来的*基于 PDB 的整合*进行对比。

一个应用后端可能需要将某些特权授予 *public*，例如 *Sys.Utl_File* 上的 *Execute*。另一方面，另一个应用后端可能会专门阻止将此特权授予 *public*。因此这两个应用后端无法共存于同一个数据库中。

- *每个应用后端的时间点恢复非常困难。* 如果发现因安装应用程序补丁而产生的应用程序代码错误导致无法挽回的数据损坏，那么通常会调用时间点恢复过程。因此，是单个应用后端而不是整个数据库需要恢复到安装错误应用程序补丁之前的那个时间点。幸运的情况下，表空间时间点恢复可能足以解决问题。但是，如果损坏的数据跨越多个表空间，时间点恢复可能会比较困难。另外，如果管理员错误地将一个或多个互相引用的表删除，时间点恢复也比较困难。
- *应用后端之间的资源管理比较困难。* 资源管理依赖人为实施的约定，在此约定下，将使用一个或多个特定服务来启动访问某特定应用后端的会话，而且没有任何服务可以用于多个应用后端。换言之，每个不同应用后端之间的差异只有人类管理员知晓，而数据库对此差异一无所知。
- *对单个应用后端的 Oracle 版本的修补无法实现。* 如果一个应用后端需要对 Oracle 数据库软件版本应用某个补丁，势必会影响所有其他应用后端。唯一的替代方法是使用数据泵将需要该补丁的应用后端迁移到已应用此补丁的另一个数据库中。但是，某些类型的数据库构件（如 XML 模式）无法通过数据泵进行迁移。
- *克隆单个应用后端比较困难。* 数据泵是唯一选择。

尽管基于模式的整合有一些明显的缺点，但是很多客户业已采用这种做法，并且已显露出明显的投资回报增长。其原因当然是高整合密度带来的资本支出节省，以及管理更少数据库带来的运营支出的节省，这些节省高于上述缺点导致的成本。

供应数据库

多年来，在大多数典型的工作日，数据库管理员需要投入大量时间来创建新数据库、将现有数据库从一台机器迁移到另一台机器上，以及出于开发、测试和问题诊断等各种目的创建现有数据库的最为当前的克隆版本。我们用术语 *供应* 表示此类琐事。

修补和升级 Oracle 数据库软件版本

虽然不像供应那样常常十分地繁琐，但是对现有数据库应用一次性补丁、捆绑补丁、补丁集更新（关键的或一般性的）和补丁集，以及将数据库从点 1 版本升级到点 2 版本，或从点 2 版本升级到下一个点 1 版本也是令人紧张且费时的的工作。我们将这类琐事统称为 *修补 Oracle 版本*。我们将会频繁地使用该词汇；

因为，如果总是对修补和升级进行区分，或者明确表明我们指的是 Oracle 数据库软件的版本，包括数据库中的可执行二进制文件和 Oracle 系统，而不是应用后端的版本，就会太过啰嗦。

Oracle Multitenant 高级描述

Oracle Database 12c 支持一种新的架构，这个新的架构允许您在单个“超级数据库”中拥有多个“子数据库”。从现在开始，我们将使用正式术语。“超级数据库”指的是 *多租户容器数据库*（缩写为 *CDB*）；“子数据库”指的是 *可插拔数据库*（缩写为 *PDB*）。换言之，新架构允许您在一个 CDB 中拥有多个 PDB。（在 12.1 中，最多为 252 个。）我们将这个新架构称为 *多租户架构*。

显然现在我们需要为原来的数据库类型，即通过 Oracle Database 11g 支持的唯一数据库类型选择一个术语。我们将其称为 *非 CDB*；并且我们将旧架构称为 *非 CDB 架构*。

Oracle Database 12c 版本 1 既支持新的多租户架构也支持旧的非 CDB 架构。换言之，您当然可以将 12.1 之前的数据库（该数据库必定是个非 CDB）升级为 12.1 非 CDB，然后就这样继续运行该数据库。但是，如果您愿意，也可以选择将 12.1 非 CDB 作为 PDB 纳入 CDB 中³。

从通过 Oracle Net 连接的客户端的角度看 PDB 就是数据库。PDB 与非 CDB 完全兼容。从现在开始，我们将这种情况称为 *PDB/非 CDB 兼容性保障*⁴。换言之，对非 CDB 无错误运行的应用后端安装方案，*无需更改*，即可在 PDB 中无错误地运行，并会获得相同的结果。连接到容纳应用后端的 PDB 的客户端代码的运行时行为与连接到容纳该应用后端的非 CDB 的客户端代码的行为完全相同。一个 PDB 只用于容纳单个应用后端。通过这种方式，PDB 提供了一种容纳应用后端的直接、声明式的方法，这样 Oracle 系统就能明确地知道各个构件属于哪个应用后端。相比之下，如果在非 CDB 中使用基于模式的整合，Oracle 系统无法知道各个构件都属于谁。我们将了解到，每个前台进程在某一时刻仅能看到一个 PDB；事实上，在最简单的使用模型中，一个 PDB 中定义的数据库用户创建的会话只能看到该 PDB — 因此，也只能看到属于单个应用后端的构件⁵。

认识到 PDB/非 CDB 的兼容性保障则可以明确肯定地回答很多问题：假如答案不是肯定的，则说明未遵循该原则。由 Oracle Corporation 工程师在整个 12.1 开发周期中执行的测试已证实与 Oracle Net 连接的客户端代码无法区分 PDB 和非 CDB。下面给出一些示例：

- *同一个 CDB 中的两个 PDB 是否能同时拥有一个名为 Scott 的用户？* 可以，因为两个非 CDB 可以同时拥有一个名为 Scott 的用户。通过扩展，同一个 CDB 中的两个 PDB 可以同时拥有：具有相同名称的角色、具有相同名称的目录、具有相同名称的版本、具有相同名称的公共同义词、具有相同名称的公共数据库链接以及具有相同名称的表空间。

3. 详细说明请见 [第 29 页](#) 上的“如何将非 CDB 用作 PDB”。

4. [第 42 页](#) 上的“ORA-65040 错误”一节中给出一条与 PDB/非 CDB 兼容性保障相关的警报信息。

5. 详细解释请见 [第 31 页](#) 上的“多租户架构的动态方面：Oracle 实例、用户和会话”。

此外，在一个 PDB 中，您可以将 *Sys.Utl_File* 上的 *Execute* 授予 *public*，而在另一个 PDB 中不进行这样的授予 — 在一个 PDB 中对 *public* 进行的授予只能在该 PDB 中看到。同样，*Any* 特权只能在授予该特权所在的 PDB 中行使。

换言之，PDB 定义全局命名空间，这和非 CDB 一样。并且它包含特权效果，这点也和非 CDB 一样。

- *可以在两个 PDB 之间创建数据库链接吗？* 可以，因为在两个非 CDB 中可以这么做。通过执行某个 SQL 语句可以创建数据库链接。因此，这样的语句在 PDB 中必须产生和在非 CDB 中同样的结果。通过扩展，您可以创建一个从 PDB 到非 CDB 的数据库链接，或者创建一个从非 CDB 到 PDB 的数据库链接 — 可以跨不同 Oracle 数据库软件版本进行创建，其方式与两个非 CDB 之间支持的方式相同。
- *可以在两个 PDB 之间建立 GoldenGate 复制吗？* 可以 — 而且通过扩展，您可以在 PDB 和非 CDB 之间，跨不同 Oracle 数据库软件版本建立复制。（此功能由支持 12.1 的首个 Oracle GoldenGate 版本提供⁶。）

从操作系统的角度看，CDB 才是数据库。每个 RAC 实例将 CDB 作为一个整体打开，而每个 SGA 可以包含来自 CDB 中每个 PDB 的数据块和库缓存结构（如子游标）⁷。（这间接地说明多租户架构与 Oracle Real Application Clusters 之间能够完全互操作。）

因此我们知道，多租户架构支持一种新的数据库内整合模型：基于 PDB 的整合。

基于 PDB 的整合模型中的 SGA 容量可以直接和基于模式的整合模型中的 SGA 容量相比。每个 RAC 实例同样可以打开非 CDB，而每个 SGA 可以包含来自非 CDB 中整合的每个应用后端的数据块和库缓存结构。因此，CDB 处于特定的 Oracle 数据库补丁集级别。而其包含的 PDB 继承了该级别，这一点与非 CDB 中的模式相同。同一非 CDB 中的 *Scott* 和 *Blake* 模式是否能具有不同 Oracle 数据库补丁集级别，这样的问题没有什么意义，同样，同一 CDB 中的两个 PDB 是否能具有不同 Oracle 数据库补丁集级别，这样的问题也没有什么意义。

具有相应特权的数据库用户允许知道其密码的人员启动一个会话，通过该会话可以看到 CDB 包含的所有 PDB 中的数据字典视图和性能视图中的信息⁸。换言之，通过 SQL 可以获得单一系统映像 — 因此通过 SQL Developer 和 Enterprise Manager 这样的工具也可以获得单一系统映像。实际上，这些工具已扩展为可以提供多租户架构实现的所有新功能。

6. 需要一些工程工作，这是因为在多租户架构中，每个重做日志条目是使用其起源所在的容器的标识符来进行批注的。请参阅第 39 页上的“Data Guard、RMAN 备份、重做和撤销”。

7. 我们将在第 34 页上的“SGA 为逻辑虚拟化”中了解到，每个 PDB 可以在每个 RAC 实例中拥有不同的 *Open_Mode*。

8. 要想充分了解此方面的内容，需要了解后面介绍的一些概念，特别是 *root*（第 12 页的“多租户架构的静态方面：水平分区的数据字典和可插拔性”）、通用用户（第 31 页的“多租户架构的动态方面：Oracle 实例、用户和会话”）以及 *container_data* 视图（第 36 页的“数据字典视图和性能视图”）。

多租户架构对 Resource Manager 功能进行了扩展，允许 CDB 级计划管理 PDB 之间的资源争用。

Oracle Active Data Guard 在 CDB 级运行，这是 RMAN 计划备份方案。在 PDB 级别支持时间点恢复。

最后，您可以从某个 CDB 拔出一个 PDB，然后将其插入到另一个 CDB 中。当然这就是取名为可插拔数据库的原因。作为对某个现有 PDB 的克隆，您还可以创建一个新的 PDB。如果底层文件系统支持精简供应，那么数 TB 的数据克隆几乎可在瞬间完成。对作为不透明实体的 PDB 进行的所有操作 — 创建全新 PDB，拔出、插入、克隆和删除 PDB — 都以 SQL 语句的方式提供。如果您请求使用精简供应的克隆，那么只需在 SQL 命令中使用关键字 *snapshot* 即可。拔出/插入操作是通过对可传输表空间技术的自然扩展实现的，而多租户架构的数据库内虚拟化特点使实现该功能成为可能。拔出/插入操作甚至支持跨不同 Oracle 数据库软件版本来执行⁹。

标准版、标准版 1 和企业版均提供这种新的多租户架构。但是，如果没有获得 Oracle Multitenant 选件的许可，您最多只能使用一个 PDB¹⁰。该模型的意义将在 [第 47 页](#) 上的“CDB 中单独的 PDB 和非 CDB”中进行讨论。

9. 所有这些内容的详细说明请见 [第 12 页](#) 上的“多租户架构的静态方面：水平分区的数据字典和可插拔性”。

10. Oracle Database 12c 许可指南正式规定了这些条款。

多租户架构的静态方面：水平分区的数据字典和可插拔性

实际上，实现了基于模式的整合的客户已经尝试在没有 Oracle 数据库内在支持的情况下实施数据库内虚拟化。本节将这种方法的困难归因于数据字典的历史架构。然后介绍多租户架构实现的内在虚拟化的基本概念：水平分区的数据字典。

表：最终的逻辑实体

静默的 Oracle 数据库只包含自己的文件。除了引导文件（控制文件、*spfile* 等），绝大多数都是实现表空间的文件。而表空间则只包含表和各种能够加速访问并确保可恢复性的结构。表是表空间存在的理由。其他各种数据库构件，如约束、视图、PL/SQL 对象等等，最终都会以表中的行来表示。表只保存三种数据：描述 Oracle 系统的元数据、描述客户创建的应用后端的元数据以及应用后端的表中的配额消耗数据。

非 CDB 架构中的单体数据字典

到 11.2 为止，描述 Oracle 系统的元数据和描述客户创建的应用后端的元数据都会在单独的一组表中进行表示，这些表统称为数据字典。众所周知，表由可执行文件来隐式维护，这些可执行文件作为 DDL 语句的附带结果实现 Oracle 实例。客户不能直接对这些表执行插入、更新和删除操作，而且它们的结构没有进行文档记录，因此只能通过数据字典视图来查询它们所表示的信息。尽管如此，客户已逐渐知晓了这些表的名称（*Obj\$*、*Tab\$*、*Col\$* 等等），并了解了它们的意义。这些数据字典表存放在专用表空间中，最值得注意的是 *System* 表空间。[图 1](#)显示了一个新创建的非 CDB 在创建任何客户构件之前的完整内容。

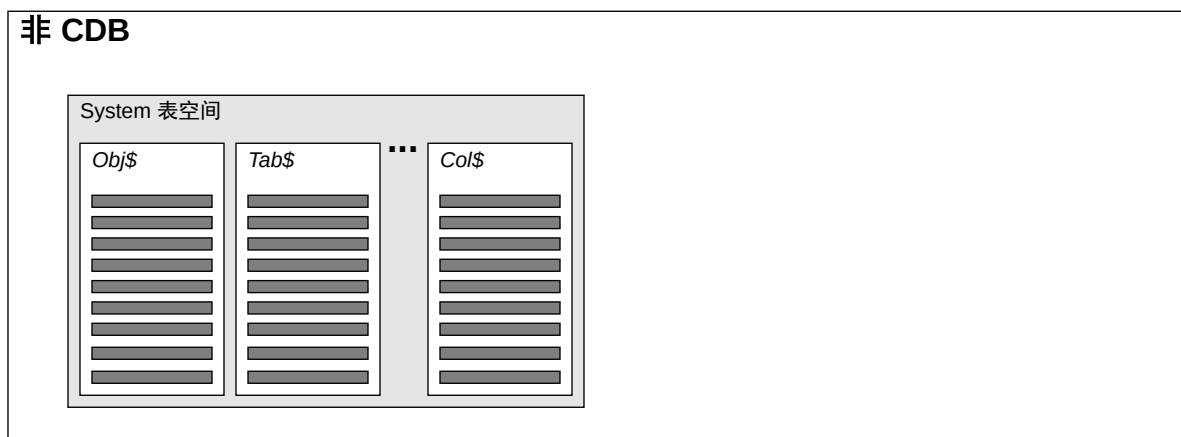


图1 一个原始的非 CDB

将一个应用后端安装到一个非 CDB 的首个效果是，一些行被插入到数据字典表中以表示该应用后端的元数据。因为应用后端的最终用途是作为应用程序的持久性机制，因此其元数据当然会包含应用程序表的描述。

按照传统的最佳实践，这些表将存储在专用应用程序数据表空间中。应用程序表很快将包含配额消耗数据，这样定义非 CDB 的所有表将成为图 2 显示的那样。

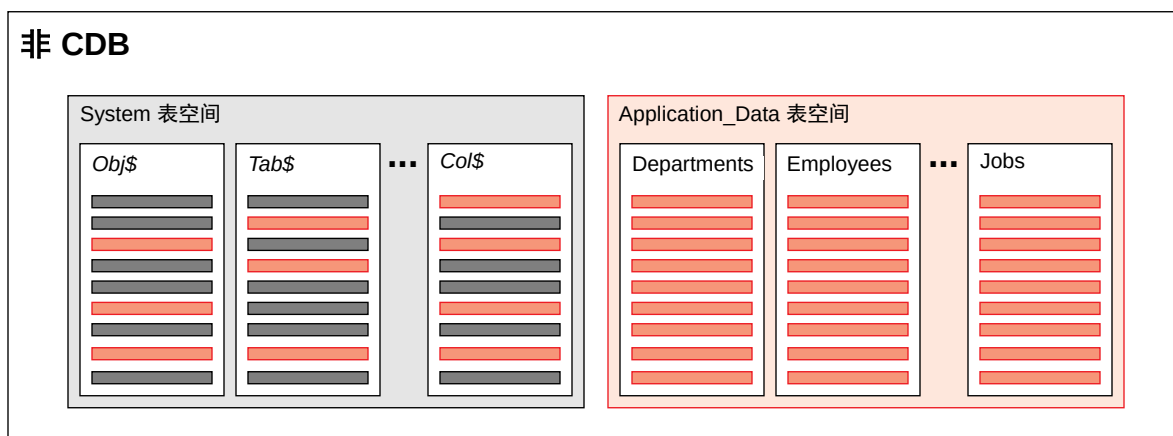


图 2 安装应用后端构件后的非 CDB

在图 2 以及后面一节中的图 3 和图 4 中，我们使用红色表示客户创建的构件（元数据行和配额消耗数据行），使用黑色表示 Oracle 系统的构件（只有元数据行）。

当然，用户、角色、版本、表空间之类的全局构件的命名空间由 User\$, Edition\$, TS\$ 等表上的唯一索引来定义。相应地，对用户和角色（包括 public 角色）进行的授权之类的全局构件，都在 Sys.Auth\$ 和 Obj.Auth\$ 表中表示。从中您立即就能明白，试图将两个或更多应用后端安装到同一非 CDB 可能带来名称冲突并随之带来安全性问题。

图 2 还能让我们了解到第二代数据泵通过使用可传输表空间带来的飞跃。第一代数据泵在导出、重放和导入时，完全使用慢之又慢¹¹ 的 SQL 结构（DDL 语句和 DML 语句）。第二代数据泵不再使用 DML 语句，而是将保存应用程序配额消耗数据（图中右侧完全以红色标识的部分）的表空间组当作能够保存自己所有表数据和索引的完备的单元，从而不再需要使用 DML 语句。最明显的是，在导入时，只使用元数据操作插入可传输表空间，因此速度超快。

图 2 还让我们了解到 12.1 之前无第三代数据泵的原因：由于每个数据字典表既包含客户元数据又包含 Oracle 系统元数据，因此用来移动应用后端这部分数据的唯一方法仍是慢之又慢的 DDL 语句。

总之，正是非 CDB 架构的单体数据字典导致基于模式的整合存在困难，并且最终方案的安全性也较低，而且应用后端的移动性也受到限制。

11. 慢之又慢 (slow-by-slow) 短语来自 Oracle 的 Tom Kyte。您会发现该词在 AskTom 和其 Oracle Magazine 文章中使用比较频繁。

换言之，非 CDB 架构的数据字典没有虚拟化。

多租户架构的水平分区数据字典

解决方案显而易见：多租户架构的数据字典实现了虚拟化。

方法概括

教科书上进行虚拟化的方法是，识别实现系统的每个对象，并用其所属系统的标识符进行标记。从概念上讲，实现虚拟化的最简单的方法是，向每个数据字典表添加一个新列以指示其描述的应用后端 — 并且对描述 Oracle 系统的行使用特定值。但是该方法无法解决应用后端的移动性问题。因此，我们通过对数据字典进行水平分区，以物理方式实现逻辑标记方案。（不要将此实现与 Oracle 分区实现混淆，该特性可供客户使用，两种实现大不相同。对数据字典进行的分区是通常的抽象意义上的分区。）图 3 显示一个包含单个应用后端的 CDB 方案。

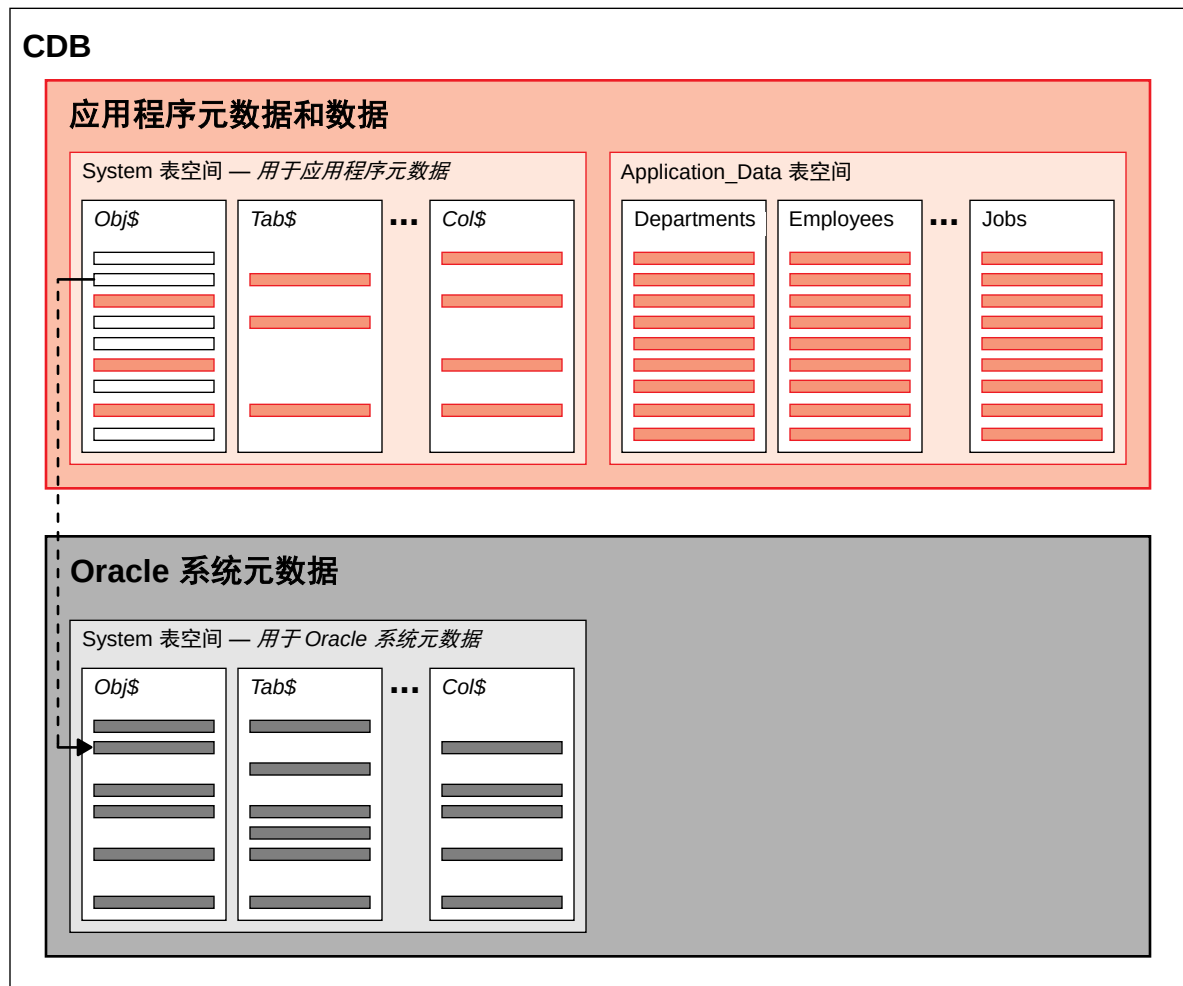


图 3 多租户架构引入水平分区数据字典

本文上图中显示的“下”半部分保存 Oracle 系统的元数据，此外再无其他元数据。“上”半部分保存应用后端的元数据，此外再无其他元数据。换言之，每个数据字典表都出现两次，一次用于 Oracle 系统，一次用于应用后端。从概念上讲，现在对数据字典表的每个查询都是对表的这两个存在的联合查询。

请注意，每组数据字典表都在自己的表空间中，因此也在自己的数据文件中。当然，数据文件的路径在文件系统中必须是唯一的。但是表空间及其包含的段的名称只需在“上”分区或“下”分区的数据字典表中保持唯一即可。因此，在两个分区中，数据字典表可以使用相同的名称，包含数据字典表的表空间也可以使用相同的名称。在下一节中将讨论这一点。

PDB 和 root 的实际定义

实现保存 Oracle 系统元数据的数据字典表的表空间组以及相应数据文件组称为 *root*。（其名称为 *CDB\$Root*。）实现保存应用后端元数据的数据字典表的表空间组（和相应的数据文件），以及保存应用后端配额消耗数据的表空间组（及其所有相应数据文件），称为 *PDB*。因此 *PDB* 是一个完整的数据库，而 *root* 只是一个元数据库。由于数据字典现已实现虚拟化，所以同一个 *CDB* 可以包含很多个 *PDB*，这种情况下，每个 *PDB* 最终都会有一组完备的属于自己的数据文件。

如 [图4](#) 所示。

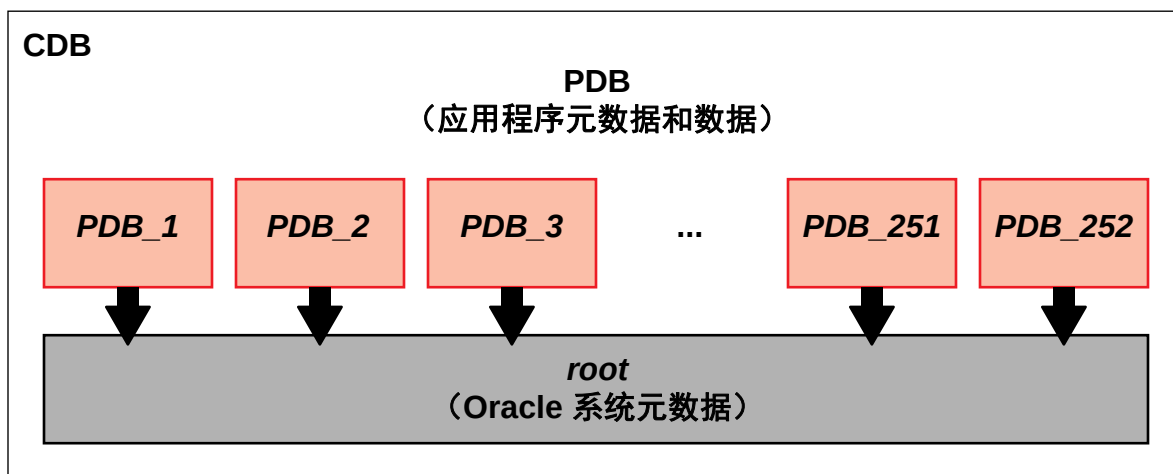


图4 在 CDB 架构中，最多可将 252 个 PDB 插入到 *root* 中

重要的是，每个 *PDB* 为用户、角色、公共同义词等所有对象定义了私有命名空间，而这些对象在非 *CDB* 的整个数据库中必须是唯一的。特定 *CDB* 中的每个 *PDB* 都可以拥有一个名为 *Scott* 的用户。因此，在 *Sys_Auth\$* 和 *Obj_Auth\$* 数据字典表中表示的授予特权和角色的结果也包含在 *PDB* 中。

尽管 *root* 与 *PDB* 明显不同，它从不保存配额消耗数据，但是这些 *CDB* 组件之间也有极其相似之处，它们都拥有数据字典，并且都可以成为前台进程的“中心”。因此我们使用术语 *容器 (container)* 作为 *root* 或 *PDB* 的超类的术语。

前台进程以及会话在其生命周期的每一时刻都拥有唯一定义的 *当前容器*。

名为 PDB 是因为可将它从一个 CDB 的 *root* 中拔出，然后插入到另一个 CDB 的 *root* 中。[图 4](#) 中的黑色大箭头表示了这种情况。当然，*root* 的元数据描述了插入其中的每个 PDB。拔出 PDB 时，将删除其对应的元数据；而在插入时，将创建描述该插入 PDB 的元数据。这是 12.1 之前的可传输表空间技术的自然扩展。因此，我们可以将拔出和插入 PDB 看做是第三代数据泵：共同完整表示应用后端的配额消耗数据和元数据可以作为一个不透明的完备单元来移动，从而无需构建和重放慢之又慢的 DDL 语句。

方法的内部概况

您可能想知道数据字典查询是如何实现的 — 但是如果您不想知道，那么可以跳过此节。我们将不再提到此处的说明，也不会依赖此部分内容来理解后续内容。

所有的查询最终会通过将块放入到缓冲区缓存来实现。因此必须给块编址，地址的一部分是存储该块的文件的编号。多租户架构使用一种基于会话的当前容器身份的相对数据文件编号方案。在一般的操作中，当前容器是 PDB，因此会话只能访问该 PDB 的数据文件中的块。这种相对数据文件编号方案在实现 Oracle 数据库的分层模块的低层实现，具体来说：会话的 PDB 标识符通过旁通道传递给低层模块；相对于非 CDB 架构，SQL 和 PL/SQL 编译与执行子系统用来向下与数据层进行通信的各种高级 API 未发生改变。这意味着，实现 Oracle 数据库的大量代码 — 整个 SQL（包括优化器）、整个 PL/SQL，以及在此基础上构建的所有内容（例如调度程序） — 在 12.1 中都无需改动，即可用于新的多租户架构。因此遵循 PDB/非 CDB 兼容性保障比较容易。

当在 PDB 环境中发出的数据字典查询需要访问描述 Oracle 系统的行时，当前容器要切换到 *root*，然后再切换回来。[图 3](#) 中从 PDB 的 *Obj\$* 表中的一行到 *root* 的 *Obj\$* 表中的一行的虚线表示发出此需求的高效机制。通过包含各种直接或间接引用 *root* *Obj\$* 表中行的细节表，在 *root* 中完全表示 Oracle 提供的对象。而在 PDB 中，只通过其 *Obj\$* 中指向 *root* 中对应行的一行来稀疏地表示该对象。

针对 PDB 实体的操作：拔出/插入、克隆、创建、删除

接下来我们将看到多租户架构中实现的这些操作如何为供应和修补 Oracle 版本提供了新的范式。

机器间的拔出/插入操作

图 5 显示 PDB_1 插入到机器 1 上运行的 CDB_1 的 root 中。按照设计，PDB_1 只容纳一个应用后端。现在的目标是将此应用后端从机器 1 迁移到机器 2 上。

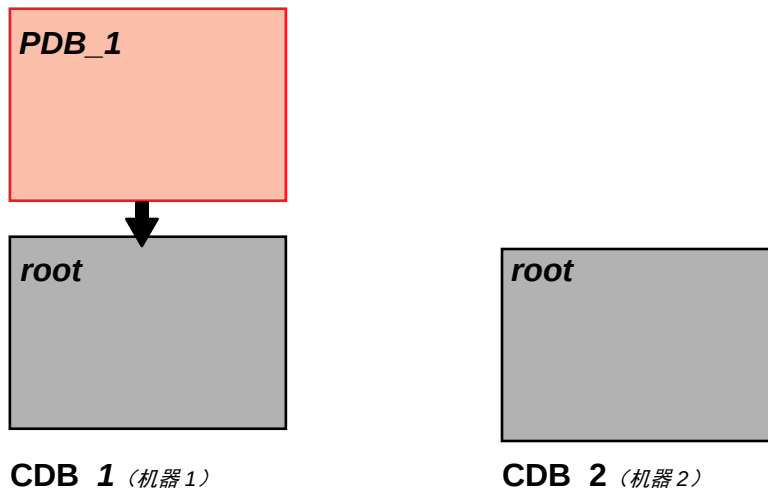


图 5 两台机器间的拔出/插入：最初 PDB_1 位于 CDB_1 中

当 PDB 包含在 CDB 中时，CDB 中保存有该 PDB 的元数据，如该 PDB 在该 CDB 中的名称及其数据文件的路径。（其中一些元数据保存在 root 中，还有一些保存在 CDB 的控制文件中。该区别目前不是讨论的重点。）拔出时，相应元数据将写入到伴随 PDB 数据文件的外部清单中。拔出操作以单个 SQL 语句来实现，如 Code_1 中所示。

```
-- Code_1
-- The PDB must be closed before unplugging it.
alter pluggable database PDB_1
unplug into '/u01/app/oracle/oradata/.../pdb_1.xml'
```

其语义是“指定该 PDB 的名称”、“表明您希望拔出该 PDB”以及“指定外部清单的路径”。

相应地，插入操作也以单个 SQL 语句来实现，如 Code_2 中所示。（拔出再插入 PDB 的 SQL 语句是 `create pluggable database` 语句的变体。其他变体还有 `clone PDB` 和创建一个全新 PDB。示例请参见 Code_2、第 25 页上的 Code_3、Code_4 和 Code_6，以

及第 26 页上的 Code_7，示例假设 Oracle 管理的文件已打开。）

```
-- Code_2
-- The PDB must be opened after plugging it in.
create pluggable database PDB_2
using '/u01/app/oracle/oradata/.../pdb_1.xml'
path_prefix = '/u01/app/oracle/oradata/pdb_2_Dir'
```

`path_prefix` 子句是可选项。它只能通过 `create pluggable database` SQL 语句来指定。对于现有 PDB，它不能改变。如果从使用该子句创建的 PDB 中发出 `create directory` SQL 语句，它将影响该语句的结果。`create directory` 的 `path` 子句中提供的字符串附加到 `create pluggable database` 的 `path_prefix` 子句中提供的字符串之后，构成新目录对象将代表的实际路径。这是控制 PDB 间隔离的重要措施。

Code_2 生成如图 6 所示的结果。

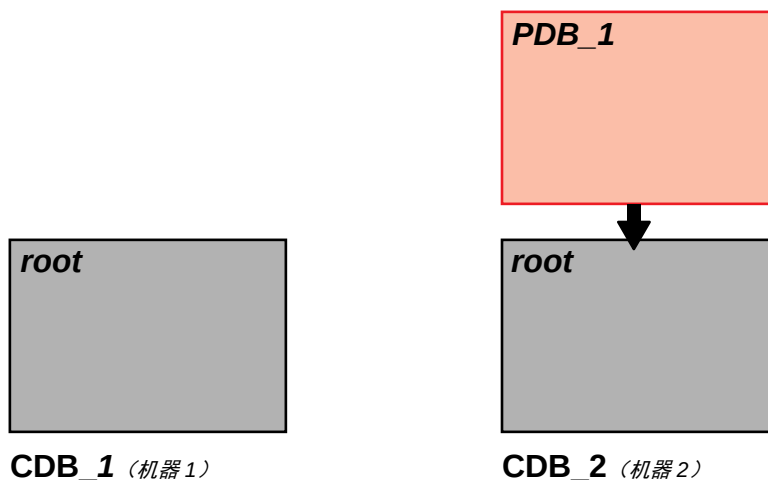


图 6 两台机器间的拔出/插入：最终 PDB_1 位于 CDB_2 中

其语义是“表明您希望获得一个新 PDB”、“表明通过指定外部清单的路径，插入已拔出的 PDB 来创建”、“在新 CDB 中给插入的 PDB 指定一个名称，确保该 PDB 中创建的任何目录对象都限制在该 CDB 使用的文件系统的指定子树下”。

跨不同操作系统、芯片组或字节序的拔出/插入

跨平台的拔出/插入需要考虑的事项与跨平台使用可传输表空间的考虑事项相同。当使用 `RMAN convert` 命令将客户创建的表空间从一种字节序转为另一种字节序时，只有块标头发生改变（因为这些标头的编码对字节序敏感）。客户创建的表中的列数据保持不变。（我们希望）大多数标头将来都使用原始标量数据类型，如 `varchar2` 和 `number` — 这样它们的表示将不会对字节序敏感，而是按照网络字节序来编码。另一方面，（我们还希望）如果使用数据类型为 `raw` 或 `blob` 的列，那么就用它们来保存知道如何解释此类数据的客户端代码使用的数据（如 `.pdf` 或 `.jpg` 文件）。但是，客户可能会以字节序敏感的方式编写数据代码，这种情况下，客户应该知道这样编码的位置，并在将包含此类数据的可传输表空间跨不同字节序平台插入后，应用自己的转换处理，这是他们的责任。

这方面的讨论比较复杂，因为拔出的 PDB 包含数据字典表，而且这些编码信息中某些列对字节序敏感。目前尚无自动转换此类列的方法。简单地说，这意味着拔出的 PDB 无法跨不同字节序平台移动。

（在数据字典保持字节序敏感的）同时，如果功能目标是跨字节序拔出/插入，那么唯一可行的方法是使用数据泵将内容迁移到目标 CDB 中的一个新的空 PDB 中。如果使用可传输表空间，那么必须使用 `RMAN convert` 对其进行处理。如果数据量比较小，那么不使用可传输表空间，使用“典型”数据泵可能速度会更快。

让我们进一步延伸字节序的讨论，如果使用了 PL/SQL 原生编译，那么特定数据字典表将包含对编译平台的芯片组敏感的机器代码。当从一个平台上运行的某个 CDB 中拔出一个包含此类机器代码的 PDB，然后将它插入另一个平台上运行的另一个 CDB 中时，如果前后两个平台具有不同的芯片组，则是不可行的。这种情况下，补救措施很简单：必须对传入 PDB 中原生编译的所有 PL/SQL 单元进行重新编译，然后才能如常使用该 PDB。

拔出/插入以修补 Oracle 版本

多租户架构支持将 PDB 从一个 CDB 中拔出，然后将其插入到具有不同 Oracle 数据库软件版本的 CDB 中。（当然，因为 12.1.0.1 目前是唯一可用的 12.1 版本，所以无法演示此功能。）

最不积极的补丁只改变 Oracle 二进制文件，换言之，按照定义，此类补丁不改变数据文件的内部内容。这意味着如果源 CDB 和目标 CDB 在此类补丁上不同，传入的 PDB 不需要任何更改；只需（当然也是必须）使用 PDB 清单中的信息检查 Oracle 数据库软件版本差别属于这种类型即可。

较积极的补丁将更改数据字典中 Oracle 系统的定义。（通常也会修改 Oracle 二进制文件。）但是既然名为补丁而不是升级，就意味着这些数据字典更改通常不会导致连锁失效。例如，如果使用新的实现对 Oracle 提供的程序包主体进行重新编译，以便修复某个错误，那么包规范依然有效，因而其相关项的闭包中的所有对象也都有效。随着 Oracle Database 11g 中细粒度相关性跟踪功能的出现，即使对 Oracle 提供的包规范进行了更改，添加了一个不重载现有子程序名称的新的子程序，那么也不会导致连锁失效。如果在非 CDB 上应用此类补丁，不会更改任何客户创建的构件。这意味着，即使是此类补丁，传入的 PDB 也不需要任何更改¹²。我们将看到，目前，即使是导致某种连锁失效的补丁，PDB 中需要的修正工作也很少。

要利用新的拔出/插入范式来最有效地修补 Oracle 版本，源 CDB 和目标 CDB 应共享一个文件系统，以便 PDB 的数据文件能够保持原地不动。如 [图 7](#) 所示。

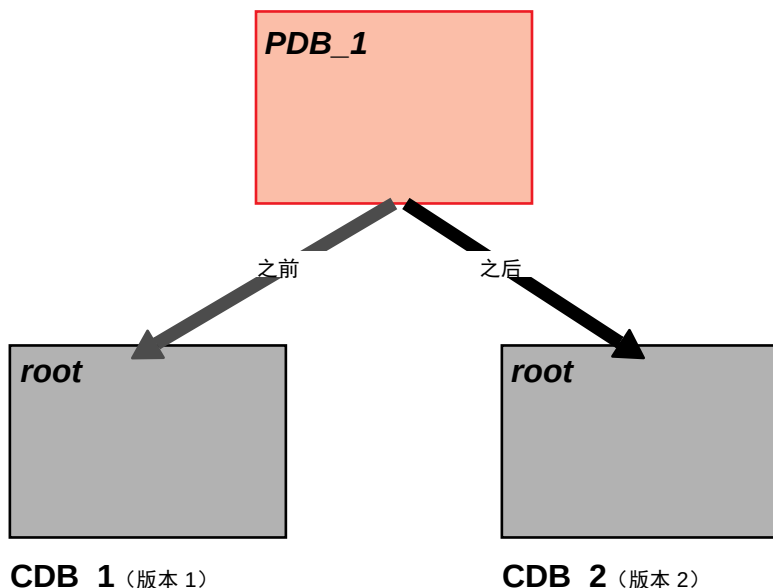


图 7 在两个 CDB 之间跨不同 Oracle 数据库软件版本执行拔出/插入操作，同时数据文件在共享文件系统上保持原地不动

因为拔出/插入操作只影响 *root* 中用于描述 PDB 的元数据，因此速度会非常快。关闭一个 PDB，将其从源 CDB 中拔出，再将其插入到目标 CDB，然后打开该 PDB，如果数据文件保持原地不动，这一系列操作在典型机器上测得的总时间为数秒 — 这是在 Oracle 数据库软件版本无差别的情况下测得的。这意味着，如果只是 Oracle 二进制文件更改上的差别，那么花费的时间将相同；而如果是数据字典更改上的差别，但不会导致任何客户创建的构件无效，那么花费的时间只稍多一点。即使补丁确实会导致客户创建的构件在某些方面无效，细粒度的相关性跟踪也会尽量减少传入 PDB 中需要进行的重新编译的任务量。请注意，我们自愿实施的兼容性规则保证修补（或升级）Oracle 版本所做的更改不会让更改之前有效的客户创建对象处于无可挽回的无效状态。重新编译总会将这些级联失效的对象返回至有效状态。

12. 如 [第 16 页](#) 上的“方法的内部概况”中所述，由于实现使用 PDB 中的 *Obj\$* 行指向 *root* 中的对应 *Obj\$* 行，而且因为 PDB 中的行需要对 *root* 中其伙伴行的签名进行编码，所以 PDB 行需要重新计算。此外，PDB 中的 *Dependency\$* 表是完整填充的。对于 Oracle 系统构件，其语义内容与 *root* 中伙伴行的语义内容相同，但是对于相关父级对象和相关对象的编号，它使用本地值。它记录细粒度相关项的详细信息，如相关父级的编译时间戳，以及相关项所依赖对象的属性的编码。您可以猜想到，一个 PDB 无法使用 *root* 中的对象编号来表达自己对 *root* 中对象的依赖，因为不同 CDB 中的编号可能不同。PDB 中的 *Obj\$* 行与 *root* 中其伙伴行之间的连接实现了两者之间的对应关系 — 如果没有此对应关系，则无法实现快速拔出/插入。因此，在插入 PDB 后，如果源 CDB 和目标 CDB 存在此类补丁上的差别，那么还需要重新计算细粒度相关项的细节，还可能需重新计算 PDB *Dependency\$* 表中的一些行。这是非常快速的操作。

Oracle 数据库软件版本 *升级* 可以进行大量数据字典更改，这会对客户创建的构件产生连锁效应。（我们用 *升级* 一词表示从点 1 版本升级到点 2 版本，如 12.1 到 12.2；或者从点 2 版本升级到下一个点 1 版本，如 12.2 到 13.1。）当对非 CDB 应用此类升级时，脚本首先处理 Oracle 系统，然后处理客户创建的构件。这意味着，如果源 CDB 和目标 CDB 存在此类升级的差别，那么，传入的 PDB 也需经历与非 CDB 中在 Oracle 系统脚本化更改之后进行的相同的脚本化更改。在传入 PDB 中进行必需更改所用的时间必然少于在非 CDB 中执行相同操作以及首先对 Oracle 系统进行必需更改所用的时间。

换言之，如果某应用后端托管在一个 PDB 中，那么使用拔出/插入方法来修补 Oracle 版本导致的应用程序中断时间 *始终* 少于将相同补丁应用到非 CDB 所导致的中断时间 — 有时中断时间可按秒来计，即使对于导致 PDB 中有限的级联失效的补丁来说也是如此。如果传入 PDB 中的内部处理保证不会导致客户创建的构件失效，那么这可以隐式执行。否则，将通知用户必须运行 Oracle 提供的脚本。

此外请注意，建立修补目标 CDB 的成本实现了高度分摊。只需一次付费，随着每个 PDB 跨不同 Oracle 数据库软件版本进行拔出/插入，将会获得修补众多 PDB 的 Oracle 版本的收益。

如果目标 CDB 的 Oracle 数据库软件版本 *早于* 源 CDB，那么拔出/插入可能出现的情况将遵循与在非 CDB 中进行补丁回退或降级完全相同的规则。（规定要求 *compatible* 参数的值必须等于早期版本的值。而且实际上多了一项要求，即没有一个客户创建的构件在语义上依赖较新的那个版本所独有的功能。）对于 Oracle 数据库软件版本的补丁回退和降级，我们将统称为 *Oracle 版本的补丁回退*。以上描述表明 Oracle 版本的补丁回退需要考虑的事项与 Oracle 版本修补相同。

最后对本节作个总结，将整个 CDB 和其众多 PDB 的 Oracle 版本的修补作为一项任务来执行，这种做法非常有效。在最不积极的一端，所有更改都仅限于 *root*。因此，修补包含 *n* 个应用后端的环境（每个应用后端托管在自己的 PDB 中，而且修补整个 CDB）需要的时间，将和修补只包含一个应用后端的环境（该应用后端托管在自己的非 CDB 中）需要的时间相同。换言之，多租户架构支持以修补非 CDB 架构中一个应用后端的 Oracle 版本的成本来修补 *n* 个应用后端的 Oracle 版本。如果开发和测试环境中的用户需要全部迁移到新版本，我们希望您首选此方法，并且之后始终只使用此方法。在生产环境中，每个整合应用后端针对新版本的认证将按照自己的计划进行，这种情况下，我们也希望您首选拔出/插入方法。

[第 24 页](#) 上的“使用 GoldenGate 复制作为拔出/插入的替代方法进行远程克隆”一节将讨论一种有效实现热拔出/插入的方法。

拔出/插入以响应 SLA 变化

新的拔出/插入范式对于快速响应 SLA 变化也非常有用。同样，源 CDB 和目标 CDB 应共享同一个文件系统，以便 PDB 的数据文件保持原地不动。

我们预计，客户通常会选择在同一个平台上安装几个 CDB，每个具有不同的 Oracle 数据库软件版本，并且进行不同的配置以满足一系列 SLA 的要求。

克隆 PDB

应用后端在其 PDB 拔出之前通常一直在使用，因而保存有配额消耗数据，而拔出的 PDB 是一切定义该应用后端的事物的完整但静止的表示。如果需要这样的正在使用中的应用后端的相同副本，即它的一个克隆副本，那么您可以拔出此 PDB，然后使用操作系统方法复制所有文件，再将原始文件插回，接着使用不同于以前的名称插入克隆的文件，并且可以插入到不同 CDB 中¹³。这种思路为 *clone PDB* SQL 语句如何工作提供了线索。不拔出源 PDB，而是由前台进程复制其文件，然后将这些文件作为克隆副本插入。如 [图 8](#) 所示。

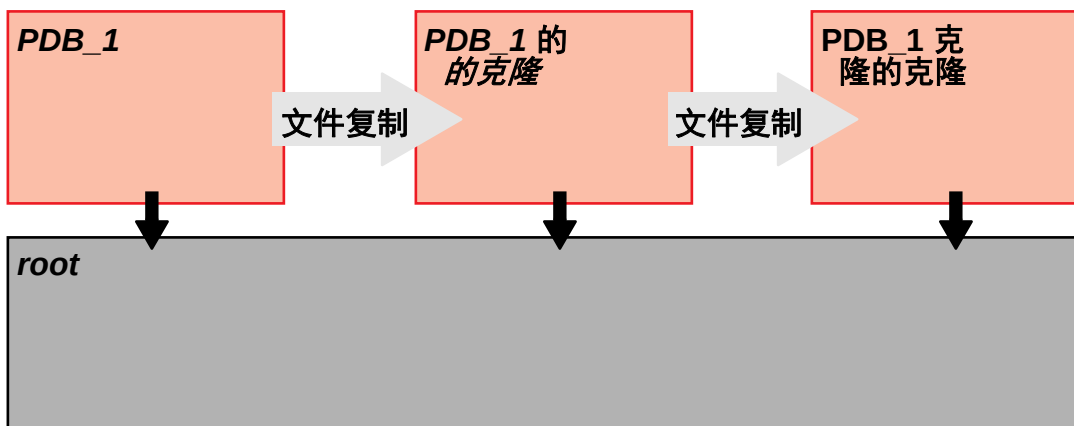


图 8 通过前台进程复制 PDB 的数据文件，以创建克隆副本

前台进程为克隆的 PDB 分配一个新的全局唯一的标识符。克隆过程会记录下其复制的源 PDB 的全局唯一标识符，这种血统可以沿袭下去，构成任意长的关系链。

多租户架构允许在源 PDB 所位于的同一个 CDB 中创建克隆的 PDB，或在不同 CDB 中执行该操作，只要两个 CDB 之间存在 Oracle Net 连接即可。我们将使用术语 *本地克隆* 和 *远程克隆* 来区分这两种选择。

13. 严格地讲，这是不健全的。*create PDB* SQL 语句为 PDB 分配全局唯一标识符，在拔出/插入操作中该标识符将一直伴随此 PDB。显然两个 PDB 不应有相同的全局唯一标识符。因此，一定不要手动复制拔出的 PDB 的文件。取而代之，在您想进行克隆时，请使用 *clone PDB* SQL 语句。（这还允许从一个拔出的 PDB 进行克隆 — 请参阅 [第 24 页](#) 上的“从拔出的 PDB 克隆”。）如果您希望得到一个“有保证的”副本，请使用 RMAN 备份或 Data Guard。

使用完整副本克隆 PDB

由于由 Oracle 数据库执行文件复制操作，因此可以使用 Oracle 并行服务器进程来减少完成此任务所需的总时间。此外，要复制的数据文件中包含 Oracle 数据块，而 Oracle 二进制文件了解这些数据块是什么。因此，并行度不受数据文件数量的限制，也不受操作系统了解的这些数据文件中粗粒度分配单元的限制，而是受数据文件组中 Oracle 数据块数量的限制。这个数字太大了，因此实际上，并行度只受可专用于 *clone PDB* 操作的 Oracle 并行服务器进程数量的限制（假设机器上有其他并发活动）。

多租户架构允许在不停止源 PDB 的情况下执行 *clone PDB* 操作 — 但是在 12.1.0.1 中不支持此功能。（此尝试将导致 ORA-65081 错误：*database or pluggable database is not open in read only mode*。）这里的难点与 RMAN 联机备份相同，即：从 t 时开始复制一组数据文件，在进行复制所需的时间段内，这些文件中的一些 Oracle 数据块一直在变化。解决方法是，如果在 t 时后，待复制的块发生了变化则发出通知，并且使用 undo 信息按照 t 时状态重构该数据块。显然，对于 *clone PDB* 克隆操作，可以重用 RMAN 方法。对于不停止源 PDB 的 *clone PDB* 操作，我们将称之为热克隆¹⁴。

使用快照复制克隆 PDB

一段时间以来，Oracle Corporation（拥有 Sun ZFS）或 NetApp 等文件系统供应商已经提供了一种方法，可在数秒内或更少时间内复制任意大的文件。该方法利用了一个普通的思想，除文件外，这还与许多其他结构相关，这个思想就是写时复制¹⁵。简而言之，对文件复制运用这一思想时，利用了这样的事实：对块的访问是通过一个单独的记录来进行的，该记录携带相应文件名称并列出指向文件块的指针。这里不是复制所有块，而是只复制指针列表，并为其指定新文件的名称。至此，源文件的指针列表和复制文件的指针列表都指向文件中的每个块。由于不使用大量存储，所以复制几乎瞬间即可完成。只有当尝试向其中一个文件中的块写入时，才会进行块的第二次复制，以允许两个文件在此块中不同。此方法的名称正是得自于这一思想。

以此方式复制大型文件时，只对一小部分块进行更改，因此时间和机器资源的净节省是非常大的。因此，在出于开发或测试目的进行克隆时，该方法非常受欢迎。通常，这种情况下，如果一个文件的大多数块保持不变，那么在相对较短的生命周期后，该文件将被删除。

文件系统供应商更喜欢“快照复制”这样的术语。而其业务优势通常被称为“精简供应”。

14. 本文包含对热克隆的讨论，这是因为如果不介绍它的话，读者会指出支持热克隆所面临的挑战和支持 RMAN 联机备份所面临的挑战一样！截至到本文的时间戳，尚不知道哪个 Oracle 数据库版本可以支持热克隆。

15. 以下 Wikipedia 文章描述了这些普通的思想：<http://en.wikipedia.org/wiki/Copy-on-write>。

客户已在使用快照复制来制作非 CDB 的 RMAN 备份集的精简供应副本，以便可以从该副本恢复被克隆的非 CDB。（需要为克隆的非 CDB 创建一个新的全局唯一标识符。）但是整个过程涉及多个不同的步骤，在多种不同的环境下执行，并且需要不同形式的授权。通常情况下，由于数据库管理员和存储管理员之间传统的劳动分工不同，因此需要多个人员，每人知晓不同的密码，他们相互协作，同步执行其工作。这种多人合作并不总是容易安排的！

在多租户架构中，前台进程向存储系统发送请求，以通过专用安全协议复制源 PDB 的数据文件。特定 CDB 在钱包中记录（针对 CDB 的数据文件的）存储系统管理员用户的用户名/密码凭证。

只有当要创建的新 PDB 与被克隆的源 PDB 位于同一个 CDB 中时，才支持快照 PDB 克隆。此外，当存在精简供应的 PDB 时，无法删除该克隆 PDB 的源 PDB。通过快照复制创建的 PDB 是无法拔出的。源 PDB 和克隆 PDB 都能以读写方式打开。

在 12.1.0.1 中，以下这些文件系统支持快照 PDB 克隆：Sun ZFS、NetApp 和 Oracle ACFS。在任意其他文件系统上尝试进行快照 PDB 克隆时，将报告错误。（此错误是 ORA-17517: *Database cloning using storage snapshot failed.*）

从拔出的 PDB 克隆

一个拔出的 PDB 可用作一个完整应用后端的非常方便的运载工具，从而完全替代早期需要运行各种工具（如 SQL*Plus、数据泵等）的方法。这种使用情况下，不是像将拔出的 PDB 插入到许多不同 CDB 中那样简单，因为这样创建的每个 PDB 会具有相同的全局唯一标识符。因此，*plug in PDB* 命令允许作为拔出 PDB 的克隆 PDB 创建一个新的 PDB，以便为其分配自己专有的全局唯一标识符。[Code_6](#) 给出了该 SQL 语句。

在该使用的一种变体中，可能会在客户站点的中心位置创建一组“最终预览版”拔出的 PDB，表示可用于压力测试或应用程序逻辑调试等用途的有用的起点。

使用 GoldenGate 复制作为拔出/插入的替代方法来进行远程克隆

如果支持热克隆，那么可以使用远程克隆在 CDB 中建立新的 PDB（否则只得通过拔出/插入方式来创建），对于应用后端托管在要移动的 PDB 中的应用程序来说，不会造成任何停机。然后，在记下热克隆启动所用的 SCN 后，可以使用 GoldenGate 复制追赶上被克隆的 PDB，然后保持跟踪。该稳定跟踪状态，与使用临时逻辑备用数据库对 Oracle 数据库软件版本进行滚动升级，具有类似的特点。当然，切换客户端应用程序的过程也是相同的。换言之，跨源 CDB 和目标 CDB 之间不同 Oracle 数据库软件版本远程克隆 PDB，与用于非 CDB 的成熟的临时逻辑备用数据库方法，在本质上极为相似。

clone PDB SQL 语句的语法

Code_3 给出基本的 *clone PDB* SQL 语句。

```
-- Code_3
create pluggable database PDB_2 from PDB_1
```

其语义是“创建 *PDB_1* 的克隆，并将新创建的 PDB 命名为 *PDB_2*”。

Code_4 给出远程克隆的 *clone PDB* SQL 语句。

```
-- Code_4
create pluggable database PDB_2 from PDB_1@CDB_02_link
```

其语义是“创建 *PDB_1* 的克隆，其所在 CDB 的 *root* 由 *CDB_02_link* 表示，并将新创建的 PDB 命名为 *PDB_2*”。（或者，数据库链接也可以表示要被克隆的 PDB。）

其中的数据库链接封装了源 CDB 位置的规范（指定监听器名称、监听器端口和服务名称，指向该数据库及其文件）。它还指定授权，以便能够启动一个会话，该会话以源 CDB 的 *root* 为其当前容器。请注意，确定位置和授权后，将使用合适的低级协议以及之前所述的 Oracle 并行服务器进程来并行传输文件。不通过数据库链接传输文件。

Code_5 给出使用快照复制的 *clone PDB* SQL 语句。

```
-- Code_5
create pluggable database PDB_2 from PDB_1 snapshot copy
```

其语义是“使用底层文件系统的精简供应方法创建 *PDB_1* 的克隆，并将新创建的 PDB 命名为 *PDB_2*”。

Code_6 给出从拔出的 PDB 进行克隆的 *clone PDB* SQL 语句。

```
-- Code_6
create pluggable database PDB 2 as clone
using '/u01/app/oracle/oradata/.../pdb_1.xml'
copy
```

其语义是“从拔出的 PDB *PDB_1* 创建克隆，将新创建的 PDB 命名为 *PDB_2*，从而将拔出的 PDB 的数据文件复制到新组中，以便能够独立于源文件进行更改”。

create PDB、*plug in PDB* 和 *clone PDB* 三个操作都是 *create pluggable database* SQL 语句的变体。因此，每个语句都支持 *path_prefix* 子句。

创建 PDB

不是从头构建定义空 PDB 的数据字典表，然后填充其 *Obj\$* 和 *Dependency\$* 表，而是在创建 CDB 时创建空 PDB。（此处，我们使用空表示不包含任何客户创建的构件。）该 PDB 称为种子 PDB，其名称是 *PDB\$Seed*。每个 CDB 必然包含一个种子 PDB；而且该种子 PDB 始终只能以只读方式打开。该种子 PDB 没有概念上的意义，它只是一个优化设备。*create PDB* 操作作为 *clone PDB* 操作的一个特例来执行。种子 PDB 的大小仅为 1GB 左右，而且在通常的机器上只需数秒即可复制它。*create PDB* 不能选择使用快照 PDB 克隆。

create PDB SQL 语句的语法

[Code_7](#) 给出基本的 *create PDB* SQL 语句。

```
-- Code_7
create pluggable database PDB_1
path_prefix = '/u01/app/oracle/oradata/pdb_2_Dir'
admin user PDB_1_Admin identified by p
roles = (DBA, Sysoper)
```

其语义是“创建一个名为 *PDB_1* 的全新 PDB，确保在该 PDB 中创建的所有目录对象都限制在 CDB 使用的文件系统的指定子树下，在新 PDB 中创建一个名为 *PDB_1_Admin* 的本地用户¹⁶ 作为它的管理员，并将给定角色列表授予该用户”。

删除 PDB

要删除的 PDB 必须处于关闭状态¹⁷。

drop PDB 操作有两种使用情况。第一种情况很明显，即当不再需要该 PDB 时删除。在非生产环境中这种情况很常见，在这类环境下，代表特定开发任务起点的 PDB 或作为分析对象的 PDB 会被反复克隆和删除。我们称之为破坏性删除。如 [Code_8](#) 中所示。

第二种使用情况不是很明显。*unplug PDB* 操作后，PDB 的信息仍在 *root* 的元数据中保留，而且其数据文件仍列在 CDB 的控制文件中。这就是为何此时使用 RMAN 备份记录该 PDB 的状态，如同其尚未拔出时一样的原因。因此，无论是否要进行备份，*unplug PDB* 操作后面必须始终跟有 *drop PDB* 操作 — 如果不进行备份，则立即删除，或者在成功完成备份后删除。当然，*plug in PDB* 操作的目的意味着必须保留数据文件。我们称之为非破坏性删除。如 [Code_9](#) 所示。

drop PDB SQL 语句的语法

[Code_8](#) 给出破坏性 *drop PDB* SQL 语句。

```
-- Code_8
drop pluggable database PDB_1 including datafiles
```

其语义是“删除名为 *PDB_1* 的 PDB，并且不留痕迹地删除其所有数据文件”。显然，应谨慎使用该语句。

16. 在 [第 31 页](#) 上的“多租户架构的动态方面：Oracle 实例、用户和会话”中我们将定义术语 *本地用户*。

17. 列 *gs\$PDBs.Open_Mode* 允许使用三种值：MOUNTED、READ ONLY 和 READ WRITE。在 [第 34 页](#) 上的“SGA 为逻辑虚拟化”中将对此进行说明。

Code_9 给出非破坏性 *drop PDB* SQL 语句。

```
-- Code_9
drop pluggable database PDB_1 keep datafiles
```

其语义是“删除名为 *PDB_1* 的 PDB，但是保留其所有数据文件”。

为什么采用 SQL 语句实现 *create PDB*、*clone PDB*、*drop PDB* 以及拔出/插入操作十分重要？

create PDB 和 *drop PDB* 操作在功能上等同于使用 *dbca* 创建或删除非 CDB — 但是要使用 *dbca*，必须首先登录到安装相应 *Oracle Home* 的机器上，而且是作为拥有相应软件安装，因而拥有在该软件安装中已创建的任何其他数据库的文件的操作系统用户来登录。因此，只有非常可靠的人员才能执行这些供应任务。对于非 CDB，*clone PDB* 操作从功能上等同于使用一系列相当复杂的步骤，这些步骤通常包括克隆 RMAN 备份集，然后从结果集恢复非 CDB。这同样要求以拥有 *Oracle Home* 的操作系统用户身份登录。对于非 CDB，与 *unplug PDB* 和 *plug in PDB* 操作在功能上最相近的对应操作是数据泵的完整数据库导出和完整数据库导入。这些操作同样要求具有 *Oracle Home* 所有者这样的操作系统授权，这样才能复制或移动转储文件。

我们回忆一下，可以使用 *unplug PDB* 和 *plug in PDB* 操作达到修补 PDB 的 *Oracle* 版本的效果。对于非 CDB，此操作同样要求具有 *Oracle Home* 所有者这样的操作系统授权。

相比之下，实现 *create PDB*、*clone PDB*、*unplug PDB*、*plug in PDB* 和 *drop PDB* 的 SQL 语句可以从客户端机器执行，且只需授权为合适的特权 *Oracle* 数据库用户即可。这些语句都是针对想要执行的操作的语义需求提供的直接原子表达式。此外，除非涉及数据文件的完整物理复制或传输，否则 SQL 语句可在数秒内执行完毕 — 这包括快照 PDB 克隆的情况。

PL/SQL 可以简单地用于实现自动化，例如将以下命令封装在一起：关闭 PDB、*unplug PDB* 以及后续的非破坏性 *drop PDB*、*plug in PDB* 以及之后的打开该 PDB。SQL Developer 和 Enterprise Manager¹⁸ 等工具直接提供 PDB 供应 SQL 语句的功能。

最终得到一个非常强大的针对数据库供应的新范式。

针对 Data Guard 保护的 CDB 执行的拔出/插入和 *clone PDB* 操作

备用数据库会自动对主数据库中的 *create PDB* 和 *clone PDB* 操作做出响应。但是对于主数据库中的 *plug in PDB* 操作，其数据文件需要经过处理之后才能在备用数据库端使用，处理方式与处理可传输表空间数据文件的方式相同。

18. Enterprise Manager 为 12.1 之前的非 CDB 提供供应功能，但是需要安装和使用服务器端代理并依赖一种操作系统授权方案。如之前所述，对于 PDB，相应功能的实现则更加直接，这些操作在 PDB 上的执行速度比在非 CDB 上快很多。

同样，在包含 *keep datafiles* 选项的 *drop PDB* 操作之后，（该操作跟在 *unplug PDB* 操作之后），必须从备用数据库环境中手动删除该拔出 PDB 的数据文件。

如何将非 CDB 用作 PDB

有两种方法：直接将一个 12.1 非 CDB 用作 PDB；以及采用非 CDB 的内容，用数据泵等工具将内容导入到空 PDB 中。

直接将 12.1 非 CDB 用作 PDB

Oracle Database 12c 既支持新的多租户架构也支持旧的非 CDB 架构。12.1 之前的非 CDB 不能直接用作 PDB，必须先使用常用的方法对其进行升级。无论从概念上还是实践上，实际直接采用都非常简单。只需关闭非 CDB，然后将其视为一个拔出的 PDB 即可。之前提到，拔出的 PDB 拥有一个由 *unplug PDB* 操作生成的外部清单。因此，在关闭要采用的非 CDB 之前，必须使其处于 `READ ONLY` 模式，然后必须执行 *DBMS_PDB.Describe()* 过程，并且作为其唯一参数指定清单的路径 — 就像使用 *unplug PDB* 操作时必须指定一个路径一样。

关闭一个要采用的非 CDB 后，很自然地要进行 *即席* RMAN 备份。

现在，可将非 CDB 的数据文件连同手动创建的清单视为普通的已拔出 PDB，简单地将其插入到目标 CDB 中，然后作为 PDB 打开即可。但是必须将 *Restricted* 状态设置为 `YES`，然后再打开它，我们将很快对此进行说明。保存配额消耗数据的表空间将立即可用，如同通过可传输表空间实现的数据泵导入一样。但是，到目前为止，原先的非 CDB 的数据字典还包含对该 Oracle 系统的完整描述。这个现在是多余的，必须删除它。这时 *Restricted* 状态仍然设置为 `YES`，此时必须运行 *noncdb_to_pdb.sql* 脚本（在 *Oracle Home* 下面的 *admin_directory* 中）。现在可以像通常那样关闭然后打开该 PDB。

我们将这种采用方法称为“升级然后插入”采用方法。

采用非 CDB 的内容

凭借 PDB/非 CDB 兼容性保障，本节介绍的方法无疑是可行的。利用数据泵的完整数据库导出和导入功能，可将大多数客户创建的构件从非 CDB 迁移到新 PDB 中。作为 12.1 中新的增强，数据泵支持完整数据库导出和完整数据库导入，从而通过各个单独的命令尽可能地利用可传输表空间。该功能称为 *完整可传输导出/导入*。该增强已经向后移植到 11.2.0.3。我们将这种采用方法称为“数据泵”采用方法。但是，某些类型的数据库构件（如 XML 模式、目录以及对 *public* 的授权）无法使用数据泵进行移动。这些构件需要使用 *即席* 方法进行迁移。

请注意，当要采用的非 CDB 相对较小，并且数据泵能够处理所有内容时，使用“数据泵”采用方法可能比使用“升级然后插入”采用方法更快速。此外，整合活动通常需要将应用后端从旧设备移动到现代设备上 — 这可能意味着跨字节序的迁移。如 [第 18 页](#)“跨不同操作系统、芯片组或字节序的拔出/插入”所述，这种情况下“数据泵”采用方法是唯一的选择。

或者，可以使用 GoldenGate 复制填充新的 PDB，之后持续跟踪源非 CDB 中的后续变化。然后可使用切换方法将客户端应用程序代码从原来的非 CDB 移动到新的 PDB 中（只造成短暂的中断），该切换方法与修补非 CDB 的 Oracle 版本时所用的临时逻辑备用数据库方法中的相关切换方法相同。

多租户架构的动态方面：Oracle 实例、用户和会话

本节探讨 RAC 实例打开的是整个 CDB 这一事实的影响。

用户、角色和授予的通用性

在一个 CDB 中，我们需要区分 *本地用户* 和 *通用用户*。之所以有两种类型的用户，原因无疑在于 PDB/非 CDB 兼容性保障。为了遵守兼容性保障，需要具备以下两个条件：

- 同一个 CDB 中的每个 PDB 都可以拥有一个（举例来说）名为 *Scott* 的客户创建的用户 — 这些用户彼此独立，只是碰巧拥有相同的名称。
- Oracle 提供的用户（如 *Sys* 和 *System*）必须存在于每个容器中，并且具有唯一定义的详细记录的用途和特征描述。

本地用户和本地角色

如第 14 页上的“多租户架构的水平分区数据字典”中所述，特定 CDB 中的每个 PDB 都可以拥有一个名为 *Scott* 的用户。这样的用户就是 *本地用户*，这种用户完全只在 PDB 的 *Users\$* 数据字典表中进行定义，因此也只为相应的 PDB 所了解。（显然，每个 PDB 中的 *Scott* 可以拥有与其他 PDB 中 *Scott* 不同的密码。）如果不具备这样的条件，则无法遵循 PDB/非 CDB 兼容性保障。相应地，*本地角色* 也完全只在特定 PDB 中进行定义。

本地用户和本地角色都不能在 *root* 中创建。

通用用户和通用角色

另一方面，对于使用特定 CDB 中某一 PDB 的会话与使用该 CDB 中任意其他 PDB 的会话来说，它们必须能看到同一组 Oracle 提供的构件。当然，这些构件包括 *Sys* 和 *System* 之类的用户，也包括 *DBA* 和 *Select_Catalog_Role* 之类的角色。*Sys* 和 *System* 是 *通用用户*，*DBA* 和 *Select_Catalog_Role* 是 *通用角色*。通用用户的名称和密码在 *root* 中定义，但是毫无二致的是 CDB 中的每个 PDB 都知道该身份 — 无论是现在还是将来 — 具有相同的名称和密码。相应地，*通用角色* 也是在 *root* 中定义并且每个 PDB 都知道它。

出于同样的原因，*Sys.DBMS_Output* 和 *System.Sqlplus_Product_Profile* 之类的对象也必须以唯一定义存在于每个容器中。这些对象称为 *通用对象*，在 *root* 中定义¹⁹。客户无法创建通用对象。

19. 在 PDB 的字典中通过 *Obj\$* 中的一行来表示通用对象。这通过名称与 *CDB\$Root* 的 *Obj\$* 中的相应行匹配（匹配 *Owner*、*Object_Name* 和 *Namespace*）。正是这种设计手段才使得 PDB 在 CDB 间拔出/插入后能够立即可用，即使在 *Dependency\$* 中的依赖图跨越 *CDB\$Root* 的通用对象和 PDB 的本地对象并且 *Dependency\$* 中的实际信息使用 *Object_ID* 值进行记录的情况下也是如此。

特权和通用角色的通用授予

如果当前容器是一个 PDB，那么授予特权或角色的结果将在该 PDB 的 *SysAuth\$* 和 *ObjAuth\$* 数据字典表中表现出来。因此，授予的结果包含在执行授予所在的 PDB 中。即使受让者是一个通用用户或通用角色也是如此。因此，一个特定通用用户或通用角色在每个 PDB 可能会得到不同的授予。但是 PDB/非 CDB 兼容性保障要求 Oracle 提供的通用用户（如 *Sys* 和 *System*）以及 Oracle 提供的通用角色（如 *DBA* 和 *Select_Catalog_Role*），在该 CDB 的每个 PDB 中具有完全相同的一组授予 — 无论是现在还是将来。（此处我们假设，客户遵循推荐的最佳实践，既不向 Oracle 提供的用户授予特权或角色，也不从这类用户撤销授予。此最佳实践不适用于客户创建的通用用户和通用角色。）该最佳实践必须得到正式的支持；否则人们可能会在某个特定的 PDB 中撤销某项特权或角色，而这将违反兼容性保障。正式支持通过 *grant* SQL 语句这种独特形式来实现。[Code_10](#) 给出一个示例。

```
-- Code_10
grant Create Session to A_Common_User container = all
```

其语义是“确保该授予项始终授予每个容器中的受让者 — 无论是现在还是将来”。在进行特权或通用角色的通用授予或撤销时，当前容器必须是 *root*。逻辑上要求每个 PDB 都必须知道该受让者 — 因此该受让者必须是一个通用用户或通用角色。此外，每个 PDB 也必须知道该授予项 — 因此它必须是一个系统特权或对象特权（由于是 Oracle 提供的，所以每个 PDB 都知道它）或是一个通用角色。

客户创建的通用用户和通用角色

对于必须能够执行 CDB 整体管理任务以及两个以上 PDB 的管理任务的人员，可以使用客户创建的通用用户作为授权媒介，这种情况下，在名称、密码和审计方面，该用户应该是一个单一实体。

该功能是 12.1 之前锁定和终止所有 Oracle 提供的用户这一完善的最佳实践的延续，不过改为依靠客户创建的用户来允许人员执行所有必需的管理任务。在 CDB 中，此最佳实践将依靠客户创建的通用用户。12.1 之前，这种用户通常并不拥有对象。然而，他们现在可以拥有对象，不过这些对象可能是，举例来说，以某种安全方式执行某些管理任务的过程。这些对象不应被视为应用后端的一部分实现。这种思想意味着，在 CDB 中，客户创建的通用用户可以拥有 *root* 中的本地对象。我们不会禁止客户创建的通用用户拥有 PDB 中的本地对象，但是不建议这样做。PDB 中的本地对象应该由本地用户拥有。

客户创建的通用角色也有类似的考虑事项。

客户创建的通用用户或通用角色的名称必须以 *C##* 或 *c##* 开头。[Code_11](#) 给出创建通用用户的语法。

```
-- Code_11
create user c##u1 container = all identified by p
```

container = all 子句也用于创建通用角色。

请注意，本地用户无法在通用用户模式中行使系统特权。

服务和会话

要通过 Oracle Net 针对某个非 CDB 创建一个会话，必须提供以下五个实际信息：监听器位置、监听器端口、服务名称、用户名和密码。只要通过前两个信息识别出某个正在运行的监听器，并且该监听器当前支持指定的服务，那么将针对其中一个打开了目标非 CDB 的 RAC 实例启动一个前台进程。此时，该前台进程以 *Sys* 身份运行。接下来，查询 *User\$* 表，查看给定用户名和密码是否有效，以及指示的用户是否拥有 *Create Session* 特权。如果这些都有效，则给定用户变成会话用户，然后客户端就可以开始发出数据库调用；否则，前台进程将终止运行，当然客户端也未连接成功。

我们在以上描述的基础上稍做添加，来描述如何针对 CDB 中的选定 PDB 创建会话。关键的一点是，要以同样的方式提供上述五个信息。当前台进程以 *Sys* 身份启动运行时，当前容器是 *root*。它能够看到目前为止收到创建会话尝试的服务的各个属性。作为 12.1 中的新特性，一个服务拥有一个指定 PDB 的属性，在完成授权后，该 PDB 将成为会话的当前容器。（如果该属性为 *null*，则表示新会话的当前容器应为 *root*。）会话现在将切换到指定容器，然后只在 *local User\$* 表中查询给定用户名和密码是否有效，以及指示的用户是否拥有 *Create Session* 特权。之后，成功/失败的情况与非 CDB 相同。

请注意，唯一的要求是，服务属性指定的容器必须知道该指定的用户。而用户是本地用户还是通用用户并不重要。

为已建立的会话更改当前容器

alter session set container SQL 语句将当前容器更改为指定目标。[Code_12](#) 给出一个示例。

```
-- Code_12
alter session set container = PDB_2
```

目标容器必须知道发出该命令的用户，并且该用户必须在目标容器中拥有 *Set Container* 系统特权。当然，发出该 SQL 语句的容器也必须知道该用户。这意味着，只有通用用户才能无误地使用该语句。该语句成功执行的结果是实现容器到容器的转换；历史信息不重要，也不会保留。

如果在一个容器中存在未完成的事务，那么您就无法在另一个容器中启动事务。

alter session set container SQL 语句只有作为顶级服务器调用才是合法的。

作为 12.1 中的新内容，*DBMS_Sql.Parse()* 的一个重载以一种严格的“trampoline”方式，在 PL/SQL 代码中提供 *alter session set container* 的结果。因此，我们将使用此函数的方法称为 *DBMS_Sql.Parse trampoline*。*Container* 这个形式参数指定解析的单个 SQL 语句的位置。当该语句终止时，当前容器自动地不加思索地重置为调用 *DBMS_Sql.Execute()* 时的状态。

即使远程执行的 SQL 语句出现错误，情况也是如此。Code_13 给出一个示例。

```
-- Code_13
begin
...surrounding PL/SQL context...
-- For example
Stmt := 'create table t(PK...)';
declare
  Cur integer := DBMS_Sql.Open_Cursor(Security_Level=>2);
  Dummy integer;
begin
  Dbms_Sql.Parse(
    c=>Cur,
    Statement=>Stmt,
    Container=>Con_Name,
    Language_Flag=>DBMS_Sql.Native);
  Dummy := DBMS_Sql.Execute(Cur);
  DBMS_Sql.Close_Cursor(Cur);
end;
...surrounding PL/SQL context...
end;
```

DBMS_Sql.Parse trampoline 只有在起始容器是 *CDB\$Root* 时才合法²⁰。

这些规则保障本地用户创建的会话无法脱离连接时所用的服务名称所指定的 PDB。这意味着，对于本地用户而言，同一 CDB 中 PDB 间的隔离模型与同一操作系统用户拥有的同一平台上非 CDB 之间的隔离模型相同²¹。

可以方便地在进行 CDB 管理的 *即席 SQL*Plus* 脚本中使用 *alter session set container*，以避免重复使用 *CONNECT* 命令，并打消对密码安全性的顾虑。对于使用 PL/SQL 编排的 CDB 管理任务而言，使用 *DBMS_Sql.Parse()* 的方法非常有用。

SGA 为逻辑虚拟化

数据库内整合具有整合密度优势，因为每个 RAC 实例的所有应用后端共享同一个 SGA。而使用基于模式的整合时，SGA 最终只是作为一个群体，尽管该群体的各项反映了所有应用后端和 Oracle 系统，但它们本身没有归类。只有关心此方面的人才会意识到是这种情况。使用基于 PDB 的整合时，每一项（特别是数据块和子游标等库缓存结构）都标注有其起源信息。换言之，SGA 是逻辑上虚拟化的。这一点很重要。数据字典是 *物理* 虚拟化的，以提供可插拔性，而 SGA 是 *逻辑* 虚拟化的，以保留数据库内整合的物理性质。如第 35 页上的图 9 所示。

20. 否则适用一般 PL/SQL 规则：代码可能是三种类型中的任意一种（匿名块、调用者权限或定义者权限），特权测试基于且仅基于当前用户。

21. 为了加强该原则，在一个 PDB 发出的 SQL 语句中无法直接引用另一个 PDB 中的对象，也无法直接引用定义对象的部分代码。我们特意决定不创造一种由三部分组成的支持跨 PDB 名称解析的命名方案（PDB 名称、模式名称、对象名称）。如果您希望从一个 PDB 引用另一个 PDB 中的对象，必须使用数据库链接。

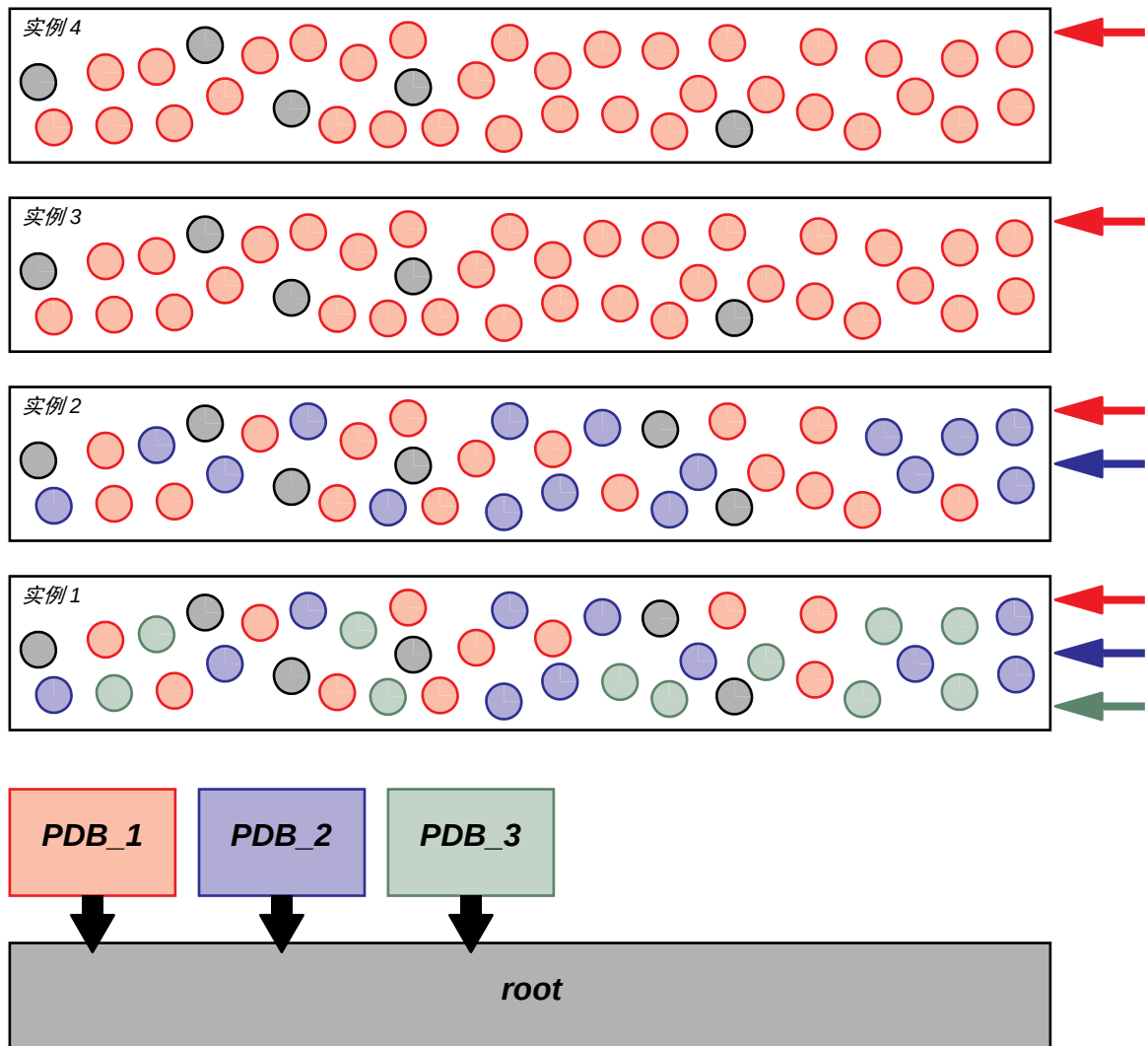


图9 PDB 和 RAC 实例的关联。右侧箭头以颜色显示每个 PDB 和每个 RAC 实例之间的关联。

使用基于模式的整合时，一些客户遵循了以下实践：将支持用户使用特定应用后端的会话与一组特定的 RAC 实例相关联。客户通过遵守自行规定的纪律来遵循上述实践，即使用特定服务来启动专用于特定应用后端的会话。

多租户架构中正式支持此规定。PDB 可以是 READ WRITE 模式、READ ONLY 模式或 MOUNTED 模式。模式显示在性能视图 `gv$PDBs.Open_Mode` 列中 — 针对每个 RAC 实例中的每个 PDB，可显式地设置为不同模式。请注意，MOUNTED 模式是 PDB 最接近于空闲状态的模式。正是由于 RAC 实例将 CDB 作为一个整体来对待，因此仅因为一个特定 PDB 而关闭整个 CDB 没有任何意义。如果当前容器是 *root*，那么可以使用 *alter pluggable database* SQL 语句对任何 PDB 设置 *Open_Mode*，而如果当前容器是某特定 PDB，那么可以使用 *alter pluggable database* SQL 语句为该 PDB 设置 *Open_Mode*。出于兼容性考虑，如果当前容器是 PDB，那么可以发出 SQL*Plus 命令 *STARTUP* 和 *SHUTDOWN*。

它们可以转换为对应的 *alter pluggable database* SQL 语句。

与 *Open_Mode* 正交的 *Restricted* 状态也可以设置为 NO 或 YES。PDB 的 *Open_Mode* 和 *Restricted* 状态有五种不同的组合。（当 *Open_Mode* 为 MOUNTED 时，*Restricted* 状态只能为 null。）如 [Code_14](#) 所示，通过使用 *alter pluggable database* SQL 语句和关键字 *force*，可将一个组合直接切换为任意另一个组合。

```
-- Code_14
-- PDB_1 starts in READ WRITE mode
alter pluggable database PDB_1 open read only force
```

在本示例中，所有连接到 *PDB_1* 并且涉及非只读事务的会话将终止，其事务将回滚。相对于非 CDB 的相应模式更改规则，这在可用性方面是一个不错的改进。

为了避免导致一些 SGA 中缓存的信息不一致，对于从 SGA 发生更改的 RAC 实例向其他 RAC 实例发送的通知，在多租户架构中只进行粗略的修剪。将打开 PDB 所在的 RAC 实例中缓存的各项发生的变化告知给未打开该特定 PDB 的 RAC 实例是没有意义的。

因此多租户架构为应用后端和 RAC 实例的关联提供了一个简单的声明式模型，相对于使用基于模式的整合时实现该目标的手工编排方案，它能够更可靠地实现该目标，并且能够提供更好的性能。我们将在[第 45 页](#)上的“PDB 和 RAC 实例的关联”中详细讨论此理念。

数据字典视图和性能视图

PDB/非 CDB 兼容性保障意味着，在当前容器是 PDB 时，如果查询数据字典视图和性能视图，那么它们必须只显示在该 PDB 中定义的构件的信息和在 *root* 中定义的构件的信息。例如，如果 *PDB_1* 中安装了 Sample Schemas，并且 *PDB_1* 是当前容器，那么 [Code_15](#) 中所示的查询

```
-- Code_15
select Owner, Object_Name
from DBA_Objects
where Owner = 'SYS'
and Object_Type like 'VIEW'
and Object_Name like 'ALL%'
union
select Owner, Object_Name
from DBA_Objects
where Object_Type like 'VIEW'
and Owner in (select Username from DBA_Users where Common = 'NO')
```


将包含以下结果：

```
SYS  ALL_ARGUMENTS
SYS  ALL_DIRECTORIES
SYS  ALL_ERRORS
SYS  ALL_INDEXES
SYS  ALL_OBJECTS
SYS  ALL_TABLES
SYS  ALL_VIEWS
HR    EMP_DETAILS_VIEW
OE    PRODUCTS
SH    PROFITS
```

但是不包含 *PDB_2* 或 CDB 中任何其他 PDB 定义的对象。

同样，当 *PDB_1* 是当前容器，并且查询 *v\$sql* 时，看到的信息只与 *PDB_1* 是当前容器时发出的 SQL 语句相关。

对于当前容器是 *root* 时查询的数据字典视图和性能视图的结果，多租户架构提供了新的规则。结果可能是所有目前打开的容器存在的被查询视图的联合。在 12.1 中由于视图有一个新的 *Con_ID* 列，所以可以区分各行。

这满足了跨整个 CDB 支持单一系统映像的业务目标要求。例如对于客户定义的命名规则实践的违规情况，可以轻松地进行监管。（一些客户坚持要求名称一定不要包含特殊字符，如单引号、双引号、分号或问号。）另一种用途可能主要是发现运行时间最长的 CDB 级 SQL 语句。

性能视图全部获得了 *Con_ID* 列。但是对于数据字典视图，只有在名称以 *CDB_* 开头的新视图系列中才会出现 *Con_ID* 列。这些视图扩展了 *DBA_* 数据字典视图的概念；*All_* 和 *User_* 风格视图未以此种方式扩展。

可以返回多个容器的行的视图（即获得新 *Con_ID* 列的视图），统称为 *container_data* 视图。如果当前容器是一个 PDB，那么每个 *container_data* 视图只显示其 *Con_ID* 值指示该 PDB 或指示整个 CDB 的行。但如果当前容器是 *root*，那么 *container_data* 视图可能显示很多容器的行。之前说过无法在 *root* 中创建本地用户，因此只有通用用户才能看到包含多个不同 *Con_ID* 值的 *container_data* 视图的结果。特定通用用户可以看到某个 *container_data* 视图中哪些容器的结果取决于该用户的一个属性（该属性通过 *Alter User* SQL 语句进行设置）。

在一个极端的情况下，如 [Code_16](#) 所示，该属性可以只为（比如说）两个指定的 PDB 指定一个特定的 *container_data* 视图。

```
-- Code_16
-- Allow c##u1 to see data for just CDB_Objects
-- in CDB$Root, PDB_1 and PDB_2
alter user c##u2
set container_data = (cdb$root, pdb_001, pdb_002)
for Sys.CDB_Objects
container = 'current'
```

请注意该 SQL 语句只在当前容器是 *root* 才能使用，而且只能在 *root* 本地设置该属性。另外请注意，*CDB\$Root* 必须始终包含在列表中。

在另一个极端的情况下，如 [Code_17](#) 所示，它可以为当前和未来的所有容器指定当前和未来的所有 `container_data` 视图（跨 Oracle 数据库版本边界）。Oracle 提供的用户，如 `Sys` 和 `System`，使用的就是这种配置。

```
-- Code_17
-- Allow c##u2 to see data for every container, present and future
-- in every container_data view
alter user c##u2
set container_data = all
container = current
```

每个 CDB 的选择和每个 PDB 的选择

整合不可避免地意味着牺牲之前独立管理应用后端享有的一些自主权，以便实现所追求的运营支出和资本支出的减低。本节首先阐述只能对 CDB 整体进行的选择，然后给出其余选择 — 可对各个 PDB 分别进行的选择的一些示例。

只能对 CDB 整体进行的选择

Oracle 数据库软件版本和平台类别

和基于模式的整合一样，集中整合的概念也是在同一个数据库中托管多个不同应用后端，从而（每个 RAC 实例）共享同一个 SGA 以及同一组后台进程。这当然意味着同一个 CDB 中的每个 PDB 必须具有相同 Oracle 数据库软件版本。*同一非 CDB 中的不同模式* 能否具有不同的 Oracle 数据库软件版本，这样的问题没有什么意义，同样，*同一 CDB 中的不同 PDB* 能否具有不同的 Oracle 数据库软件版本，这样的问题也没有什么意义。这两种数据库内整合模型当然都意味着所有整合的应用后端都只能选择同一种平台类型和操作系统类型以及版本。

而用于修补 Oracle 版本的拔出/插入范式消除了由于 CDB 只能整体采用一个特定 Oracle 数据库软件版本而导致的不便之处。此外，该范式带来的移动性放松了基于模式的整合在应用后端和托管应用后端的非 CDB 的运行平台、操作系统类型和版本之间建立的绑定关系。

Spfile、控制文件和密码文件

这些文件通常用于整个 CDB。这意味着 PDB 实际无法拥有自己的 *spfile*。在[第 42 页](#)上的“PDB 可设置的初始化参数和数据库属性”中，我们将看到，在 PDB 范围内，可使用 *alter system* 持久性地设置一些参数值。这些参数值将以适当的方式持久保存 — 不过实际不是放在 *spfile* 中。进一步说，当您打开一个 PDB 时，您无法指定 *pfile*。因此，如果您希望转储在 PDB 范围内通过 *alter system* 专门设置的参数值，那么您必须通过编写 SQL 查询来完成。相应地，要强制使用这些值，您必须编写一系列 *alter system* SQL 语句。

Data Guard、RMAN 备份、重做和撤销

使用基于模式的整合的客户在运营支出上已呈现出明显的减少，因为他们可以针对一个托管多个不同应用后端的非 CDB 设置和运行 Data Guard — 而不是像整合之前那样，将每个应用后端的 Data Guard 的管理作为单独任务来执行。这是多合一管理原则的典型示例。同样，通过运行单个计划好的惠及众多不同应用后端的 RMAN 备份方案，这些客户也实现了运营支出节省。不过，如[第 5 页](#)上的“努力实现最大的整合密度”所述，对于应用后端时间点恢复，基于模式的整合无法提供一个通常可行的解决方案。这个问题我们将在[第 42 页](#)上的“PDB 的即席 RMAN 备份”中讨论。

Data Guard 和 RMAN 备份都依赖重做日志：Data Guard 持续在备用站点应用重做；而 RMAN 备份在从备份时间点前滚恢复至所需恢复时间点时使用重做。因此，为了让基于 PDB 的整合能够实现与基于模式的整合一样的多合一管理节省，CDB 整体应该拥有单个重做流。这意味着，和 SGA 的情况一样，重做日志是**逻辑**虚拟化的：每个重做日志条目都标注有容器标识符以指示其记录的更改所发生的位置。重做和撤销密切相关，因此对于每个 RAC 实例，CDB 整体拥有单个撤销表空间。而撤销也是**逻辑**虚拟化的：每个撤销记录同样标注有原始容器标识符。对于性能影响的讨论有以下两点：

- 首先，使用基于 PDB 的整合与使用基于模式的整合在这一方面情况相同。客户已经证明基于模式的整合不会影响性能。在全局范围中，其他一些因素也在起作用 — 最值得注意的是，当使用这两种数据库内整合模型中的任意一种时，为应用后端集提供服务所需的后台进程总体上将少得多；相对于每个应用后端各自拥有大大小小不同限制的 SGA，所有应用后端联合在一起共享一个 SGA 将能够非常有效地利用内存。
- 其次，新的 within-redo 和 within-undo 索引方案能够加快对相关 PDB 记录的访问速度。此外，由于重做和撤销各自逻辑虚拟化“分区”在功能上的独立性，多个并发进程实际可以同时访问这些分区而不用相互等待。

旨在通过整合提高投资回报的客户通常对自己的整合候选对象使用具有代表性的模型来完成自己的实证调研。

字符集

多租户架构允许每个 PDB 拥有自己的字符集。但是，这样也会削弱多合一管理的优势。

此外，最近几年出现了一种趋势，即大量应用程序拥有国际范围的多语言用户群。越来越多的客户端处理使用 Unicode 来执行这一事实就反映了上述趋势。如果数据不是以 Unicode 形式存储在数据库中，那么在客户端和数据库之间将来回进行数据转换，而这意味着效率低下。

因此我们决定只允许对 CDB 整体选择字符集。我们建议使用 Unicode (*AL32UTF8*) 作为 CDB 的字符集。

当然，还要考虑到其应用后端使用特定字符集而且又无法经济地进行重新设计的原有应用程序这种情况。在这种情况下，可以利用这样的事实：您可以在同一个平台上拥有多个 CDB，而每个 CDB 使用自己的字符集。（这只是一个更为一般性的原则的一个示例。在同一个平台上拥有多个 CDB 的最明显的原因是允许每个 CDB 拥有不同的 Oracle 数据库软件版本。）

CDB 级初始化参数和数据库属性

在 12.1 中，`v$System_Parameter` 视图系列拥有一个列，`IsPDB_Modifiable`，该列允许值为 `FALSE` 和 `TRUE`。（我们将在第 42 页上的“PDB 可设置的初始化参数和数据库属性”中讨论 `IsPDB_Modifiable TRUE` 的情况）。约有 200 个参数或属性拥有 `IsPDB_Modifiable FALSE`。下面是一些说明性示例：

```
audit_file_dest
audit_trail
background_core_dump
background_dump_dest
core_dump_dest
cpu_count
db_block_size
db_name
db_recovery_file_dest
db_recovery_file_dest_size
db_unique_name
instance_name
local_listener
log_archive_dest
log_archive_duplex_dest
memory_max_target
memory_target
parallel_degree_policy
pga_aggregate_limit
pga_aggregate_target
processes
result_cache_max_size
sga_max_size
sga_target
shared_pool_reserved_size
shared_pool_size
undo_management
undo_retention
undo_tablespace
user_dump_dest
```

此列表针对多合一管理主题。

如果当前容器是一个 PDB，除了已经讨论过的 `NLS_CHARACTERSET`，`Database_Properties` 视图中列出的所有其他属性都可以通过 `alter database SQL` 语句进行设置。

AWR 报告

您可以为整体 CDB 创建 AWR 报告，我们希望在层级上运行调度的报告。为支持在单个应用后端范围内的即席性能分析，您还可以为特定 PDB 创建 AWR 报告。

可对各个 PDB 分别进行的选择

PDB 时间点恢复

每个 PDB 最显著的自由是能够在不影响同一个 CDB 中共存的 PDB 的可用性的情况下执行 PDB 时间点恢复。此能力是基于模式的整合与基于 PDB 的整合之间最为显著的区别之一。前者没有合适的方法可以满足每个应用后端时间点恢复这个业务目标。12.1 中不支持 `flashback pluggable database` 命令。

PDB 的即席 RMAN 备份

如之前所述，为了最好地遵循多合一管理原则，通常需要在 CDB 级执行调度的 RMAN 备份。但是，有时也需要对特定 PDB 进行即席备份。我们将在第 26 页上的“删除 PDB”中结合 `unplug PDB` 命令和 `drop PDB` 命令（保留数据文件）的联合使用来讨论这一点。

`alter system flush Shared_Pool`

如第 31 页上的“多租户架构的动态方面：Oracle 实例、用户和会话”中所述，块缓冲和库缓存结构中的数据块使用容器标识符来进行标注。如果当前容器是一个 PDB，这会直接影响 `alter system flush Shared_Pool` 的结果。

PDB 可设置的初始化参数和数据库属性

`v$System_Parameter` 视图中 `IsSes_Modifiable` 和 `IsSys_Modifiable` 都不等于 `FALSE` 的每个初始化参数，在当前容器是一个 PDB 时，都可以使用 `alter system` 进行设置。换言之，所有这些参数的 `IsPDB_Modifiable` 都等于 `TRUE`。此外，其他一些 `IsSys_Modifiable` 不等于 `FALSE` 而 `IsSes_Modifiable` 等于 `FALSE` 的参数，也可以在 PDB 范围内使用 `alter system` 进行设置。在 12.1 中这是一个相对较小的列表：

```
cell_offload_decryption
fixed_date
listener_networks
max_string_size
open_cursors
optimizer_secure_view_merging
resource_limit
resource_manager_plan
sessions
```

作为对 `IsSys_Modifiable` 的其他允许值 `IMMEDIATE` 和 `DEFERRED` 的反映，`scope` 子句（`scope=memory`、`scope=spfile` 或 `scope=both`）具有以下预期效果：值可以根据要求持久保存。为了遵循 PDB/非 CDB 兼容性保障，保留了成熟的 `alter system` 语法 — 但是持久性机制实际上不是 `spfile`。而是将值保存在合适的数据字典表中，以便这些 PDB 特定值既能在 PDB 打开时生效，又能随着拔出/插入操作一直伴随 PDB。

ORA-65040 错误

Oracle 数据库支持非常广泛的 SQL 语句。其中的一些语句只能由控制整个数据库的管理员在特殊情况下使用。在 CDB 中，有一小部分只能由管理员使用的 SQL 语句无法在 PDB 中发出。如果尝试在 PDB 中发出这样的语句将导致 `ORA-65040 operation not allowed from within a pluggable database` 错误。这在很大程度上是设计所为。遇到此情况时，应阅读 PDB/非 CDB 兼容性保障。请注意，所有 SQL 语句都可以从客户端代码自动发出。但是称得上应用程序的客户端代码 — 而不是类似 Enterprise Manager 这样的管理工具 — 不会发出导致 `ORA-65040` 错误的语句。即使是称作安装自动化的客户端代码，如果遵循最佳实践，也不会发出这样的语句。

CDB 内 PDB 间的资源管理

使用同一平台上托管的不同应用后端的会话之间将争用以下这些计算资源：

- 并发会话的数目
- CPU
- 使用 Oracle 并行服务器进程的能力
- 文件 i/o
- 使用 SGA 内存以及分配 PGA 的能力
- 网络 i/o

一些客户希望全权控制对所有这些资源的争用，因此将每个应用后端托管在其各自的虚拟机上的非 CDB 中，从而能够利用操作系统的虚拟化。但是，这种模式会严重影响统一管理运营开销的节省。许多客户已发现，通过将多合一管理节省这一目标放在首位，将会最大限度地提高投资回报，因此使用了基于模式的整合。

在基于模式的整合环境中，可以使用 Resource Manager。但是需要一种仔细规划、人为监管的纪律，即对各个应用后端的访问只能分别通过专门创建的服务来进行。基于模式的整合存在的问题是，只有工作人员知道如何区分每个不同应用后端的边界，而 Oracle 数据库对此一无所知。而使用基于 PDB 的整合时，PDB 提供了强大的声明式机制，工作人员能够通过该机制告诉 Oracle 数据库边界在哪里：每个应用后端安装在自己的 PDB 中，任何一个 PDB 只能安置一个应用后端。

在多租户架构中对 Resource Manager 进行了增强，提供了创建 CDB 级计划的功能，以便对 CDB 中各 PDB 间的资源争用加以控制。

在 12.1 中由 CDB 级计划控制的计算资源

在 12.1 中，Resource Manager CDB 级计划控制以下资源：

- 并发会话的数目
- CPU
- 使用 Oracle 并行服务器进程的能力
- 文件 i/o — 但仅限 Exadata 上

但是该计划不控制以下资源：

- 使用 SGA 内存以及分配 PGA 的能力

- 网络 i/o

PDB 和 RAC 实例关联²²可以作为一种粗粒度的方法，用于控制对 SGA 和 PGA 内存的争用。

份额和上限模型

Resource Manager 基于两个概念实现了一种行业标准模型：*份额 (share)* 和 *上限 (cap)*²³。在此模型中，每个竞争对手各分得一定数量的资源份额，数量为 1 到任何正整数（但是如果超过 10 则是无益的）。在某一时刻，通常只有一部分竞争对手处于活动状态。每个活动的竞争对手获得 n/t 的托管资源，其中 n 是分配给特定竞争对手的份额数量， t 是分配给所有当前活动竞争对手的份额总数。可以选择性地给任何竞争对手赋予一个上限，这是一个范围在 0 到 1 之间的系数（通常以百分比表示）。如果一个竞争对手的上限为 c ，那么它获得的托管资源比例从不会超过该系数，无论份额计算结果如何都是如此。当然，具有上限的竞争对手可能会获得更少的资源，具体取决于份额计算的即时结果。

份额概念显然十分重要，它是资源管理的要素。当竞争对手的数量逐渐增加至某一规划的数量时，上限概念的重要性也会体现出来。它确保给最终计划预留空间，而不会在过程中设定错误的预期。（性能改进会让用户非常高兴——当一个应用程序从一台机器迁移到另一台更强大的机器上，不设定上限，作为该机器最初唯一托管的应用程序时，您将看到性能上的改进。但是众所周知，这种情况不会持续很久。随着越来越多的应用程序迁移到该整合机器上，用户会发现性能在不断变差。因此，最好限定整合完成时应用程序的预期性能等级，这是将应用程序迁移到整合机器的首要考虑事项之一。）

对于 PDB 间的资源管理，在创建 PDB 时，默认情况下只为其分配一份资源，并且不设定上限。

12.1 中 CDB 级计划管理会话、CPU、Oracle 并行服务器进程以及文件 i/o 的方式

设置计划或修改计划时，其效果会很快被所有当前会话感受到²⁴。

22. 请见第 45 页“PDB 和 RAC 实例的关联”中的介绍。

23. 下面是 1995 年发表的一篇有关该主题的学术论文：

http://people.cs.umass.edu/~mcornier/courses/691J/papers/PS/waldspurger_stride/waldspurger95stride.pdf

下面是一家操作系统虚拟化软件供应商在开发人员论坛上的发帖：

<http://communities.vmware.com/docs/DOC-7272>

上限概念有时也称为资源利用限制。

24. 可以使用 `alter system set resource_manager_plan = My_Plan` 设置计划。如第 42 页“PDB 可设置的初始化参数和数据库属性”中所述，可以在容器范围内设置 `resource_manager_plan` 初始化参数。因此，要设置 CDB 级计划，在当前容器为 `root` 时发出 `alter system` 命令。

并发会话的数量只受资源上限的限制，此时份额概念无意义。实际上可使用 *sessions* 初始化参数设置会话上限²⁵。

上述的份额和上限机制可用于管理 CPU 和文件 i/o。每 100 毫秒对会话进行一次细粒度的调度。

Code_18 显示如何配置 CDB 级计划，以便为 *PDB_1* 分配一份资源，为 *PDB_2* 分配三份资源。

```
-- Code_18
DBMS_Resource_Manager.Create_CDB_Plan(
  'My_Plan', 'some comment');

DBMS_Resource_Manager.Create_CDB_Plan_Directive(
  'My_Plan', 'PDB_1', Shares => 1);

DBMS_Resource_Manager.Create_CDB_Plan_Directive(
  'My_Plan', 'PDB_2', Shares => 3);
```

Code_19 显示如何修改一个 CDB 级计划，以便为 *PDB_1* 分配两份资源，为 *PDB_2* 分配五份资源，并将它们的上限分别设置为 20% 和 50%。

```
-- Code_19
DBMS_Resource_Manager.Update_CDB_Plan_Directive(
  'My_Plan', 'PDB_1', New_Shares => 2);

DBMS_Resource_Manager.Update_CDB_Plan_Directive(
  'My_Plan', 'PDB_1', New_Utilization_Limit => 20);

DBMS_Resource_Manager.Update_CDB_Plan_Directive(
  'My_Plan', 'PDB_2', New_Shares => 5);

DBMS_Resource_Manager.Update_CDB_Plan_Directive(
  'My_Plan', 'PDB_2', New_Utilization_Limit => 50);
```

使用 Oracle 并行服务器进程的能力也由上述份额模型进行控制。*parallel_degree_policy* 初始化参数必须设为 *AUTO*。这样可以让足够多的 SQL 语句并行运行以使机器保持忙碌，而且可通过 SQL 语句排队避免并行度降级。这方面，上限概念有自己单独的参数化方法，即在 *DBMS_Resource_Manager* 的 *Create_CDB_Plan_Directive* 子程序中使用参数 *parallel_server_limit*，而在 *Update_CDB_Plan_Directive* 子程序中使用参数 *new_parallel_server_limit*。

PDB 和 RAC 实例的关联

如第 31 页上的“多租户架构的动态方面：Oracle 实例、用户和会话”中所述，在每个 RAC 实例中，每个 PDB 的 *Open_Mode* 可分别设置为 *MOUNTED*、*READ ONLY* 或 *READ WRITE*，因而可能不同。例如，如果将一个包含 8 个 PDB 的 CDB 配置为一个包含 8 个 Oracle 实例的 RAC 数据库，让每个 PDB 只能在一个 RAC 实例中打开，因此每个 RAC 实例也只能打开一个 PDB。此方案提供尽可能最高程度的资源隔离。但是这牺牲了对 SGA 和后台进程的共享，共享会带来高整合密度，而整合密度又是基于 PDB 的整合（以及基于模式的整合）的重要标志。

25. 如第 42 页“PDB 可设置的初始化参数和数据库属性”中所述，如果当前容器是一个 PDB，那么可使用 *alter system* SQL 语句设置 *sessions* 这个初始化参数。

也许像下面这样使用 PDB 和 RAC 实例关联更合理：对于一个拥有数百或更多 PDB 的 CDB，在进行关联时，将前四分之一（在处理能力需求方面）PDB 与 75% 的 RAC 实例相关联，其余四分之三 PDB 与其余 25% 的 RAC 实例相关联。

CDB 中单独的 PDB 和非 CDB

假设一个应用后端有很高的吞吐量需求，因而需要一个特定平台专供其使用，那么它可以安装在一个非 CDB 中，或者安装在（除了种子 PDB 之外的）一个 PDB 中，而该 PDB 是所在 CDB 中的唯一 PDB。我们将后一种配置称为*单独的 PDB (one-PDB)*，也称为*单租户配置*。

非 CDB 是否在某方面比单租户配置更好呢？

我们十分肯定答案为不。PDB/非 CDB 兼容性保障表明它们之间没有功能上的差异。Oracle Corporation 的工程师经过仔细测试后也证实了这一点，而且还证实没有性能上的差异。

反过来另一个更有趣的问题是：单租户配置是否比非 CDB 更好呢？

对此答案为是。即使您使用的 PDB 永不超过一个，新的多租户架构也会为您带来如下明显的优势：

- 在功能方面，拔出/插入操作为您提供第三代数据泵功能。
- 拔出/插入到新建的空 CDB 中的操作，将为您提供一种修补 Oracle 版本的新范式²⁶。

基于这种认识我们决定将 Oracle Multitenant 选件设计为在特定 CDB 中支持 2 到 252 个 PDB。（支持的 252 个 PDB 中不包含种子 PDB。）

在标准版、标准版 1 和企业版中可以使用单租户配置，进而可以选择使用多租户架构，无需额外费用²⁷。

26. 建议热克隆（如果未来版本支持此功能）采用以下方法：热克隆到一个具有相同 Oracle 数据库软件版本的“临时”CDB 中，记录下创建时使用的 SCN，然后拔出/插入到更新版本的 CDB 中，之后使用 GoldenGate 复制追赶上源 PDB 并保持跟踪。这样做在功能上等同于使用临时逻辑备用数据库，并且可用性方面有很大改进。

27. Oracle Database 12c 许可指南正式规定了这些条款。

总结

我们已经看到旧的非 CDB 架构和新的多租户架构之间的本质区别是后者能够提供真正的数据库内虚拟化。这种数据库内虚拟化是通过水平分区在数据字典中以物理方式实现的，所谓水平分区，就是将 *root* 中的 Oracle 系统，与 PDB 中的客户系统相分离 — 这种基本分离进而支持在同一个 CDB 包含多个 PDB。PDB 是一种描述整合应用后端的声明式机制。

真正的数据库内虚拟化消除了基于模式的整合的以下原则性缺点：全局名称的冲突，这种冲突意味着必须首先对现有应用后端进行代价高昂且具有风险的更改，然后这些应用后端才能在同一个非 CDB 中共存；以及 *any* 特权和类似的强大特权、对 *public* 的授予跨越非 CDB 的所有应用后端。

root 和 PDB 之间的物理分离带来了可插拔性。而可插拔性事实上通过拔出/插入提供了第三代数据泵功能。此外，具有不同 Oracle 数据库软件版本的 CDB 间的拔出/插入还提供了一种新的修补 Oracle 版本的范式。可插拔性消除了基于模式的整合存在的以下两个问题：供应困难（即将应用后端从一个地方迁移到另一个地方，以及克隆应用后端）；如果某些情况下需要修补一个应用后端的 Oracle 版本，同时要求不影响环境中其他不准备进行更改的应用后端，那么这是无法实现的。

我们也看到，对于基于 PDB 的整合与基于模式的整合，其 SGA 和后台进程共享模型基本是相同的 — 因此完全保留了原有方法的高整合密度优势。此外 SGA 中的逻辑虚拟化（通过数据块携带到数据文件中）、重做和撤销为 CDB 内 PDB 间的资源管理提供了新的能力，并支持 PDB 时间点恢复。

最后，可以这么说，CDB 与 PDB 的关系如同操作系统与非 CDB 的关系。而 PDB/非 CDB 兼容性保障意味着，体现多租户架构特征的新现象是 *root*。与各种操作系统原语编排供应任务以及其他针对非 CDB 的维护任务一样，针对 *root* 执行的 SQL 语句会执行对 PDB 的相应任务。这进而意味着，所有数据库管理员所熟知的、独立于平台的、只要 Oracle 数据库可用就可以使用的、可移植的 PL/SQL，成为在 PDB 上实现操作自动化的语言。

因此 Oracle Multitenant 代表新一代的应用后端整合。它提供基于模式的整合的采用者一度希望获得的多合一管理优势。采用它仅需部署方面的工作，应用后端和客户端代码都无需更改。

附录 A：Oracle 数据库文档库中有关多租户架构的介绍

有关简介和概述，请从“概念”一书开始了解。其中包含一个新的主要章节，即 *多租户架构*。

主要内容在管理员指南中给出。其中也包含一个新的主要章节，即 *管理多租户环境*。由于 PDB/非 CDB 兼容性保障，数据库开发指南只需顺便提及多租户架构。

（一个版本的作用域是在其中定义该版本的 PDB。这意味着多个并发 EBR 操作可以在一个 CDB 中执行。对于使用基于模式的整合在同一个 12.1 之前版本的数据库（必然是非 CDB）中实现多个应用后端的客户来说，当其中多个后端需要同时进行联机修补时，这些客户会面临挑战，而上述事实则消除了这种挑战。）

安全指南中将介绍本地用户和通用用户之间的新差异、对当前容器的权限的作用范围界定，以及这如何引出新的概念，即对超级数据库管理员（CDB 管理）和应用程序管理员（PDB 管理）之间的职责分离的正式控制。此书还将介绍该领域的另一个基本概念：*container_data* 视图。

多租户架构引入了一些值得注意的新的 SQL 结构。这些结构主要限于对 PDB 实体执行操作。SQL 语言参考中给出了这些结构的语法和语义的正式定义。

提供了一些全新的数据字典视图（如 *DBA_PDB*）和性能视图（如 *v\$PDB*）。比较明显的是，每个性能视图都有一个新的 *Con_ID* 列 — 容器标识符，用于指出该视图展示的实际信息的出处。相应地，不过使用的是略微不同的模型，对数据字典视图（*DBA_*、*All_* 和 *User_*）的风格集合进行了扩展，增加了新的 *CDB_* 风格。每个 *CDB_* 视图都有一个新的 *Con_ID* 列。视图中 *CDB_* 列的存在反映出该视图是一个 *container_data* 视图。所有这些视图在数据库参考中进行介绍。

甲骨文（中国）软件系统有限公司

北京远洋光华中心办公室

地址：北京市朝阳区景华南街5号远洋光华中心C座21层
邮编：100020
电话：(86.10) 6535-6688
传真：(86.10) 6515-1015

北京汉威办公室

地址：北京市朝阳区光华路7号汉威大厦10层1003-1005单元
邮编：100004
电话：(86.10) 6535-6688
传真：(86.10) 6561-3235

北京甲骨文大厦

地址：北京市海淀区中关村软件园24号楼甲骨文大厦
邮编：100193
电话：(86.10) 6106-6000
传真：(86.10) 6106-5000

北京国际软件大厦办公室

地址：北京市海淀区中关村软件园9号楼国际软件大厦二区308单元
邮编：100193
电话：(86.10) 8279-8400
传真：(86.10) 8279-8686

北京孵化器办公室

地址：北京市海淀区中关村软件园孵化器2号楼A座一层
邮编：100193
电话：(86.10) 8278-6000
传真：(86.10) 8282-6401

上海名人商业大厦办公室

地址：上海市黄浦区天津路155号名人商业大厦12层
邮编：200001
电话：(86.21) 2302-3000
传真：(86.21) 6340-6055

上海腾飞浦汇大厦办公室

地址：上海市黄浦区福州路318号腾飞浦汇大厦508-509室
邮编：200001
电话：(86.21) 2302-3000
传真：(86.21) 6391-2366

上海创智天地10号楼办公室

地址：上海市杨浦区淞沪路290号创智天地10号楼512-516单元
邮编：200433
电话：(86.21) 6095-2500
传真：(86.21) 6107-5108

上海创智天地11号楼办公室

地址：上海市杨浦区淞沪路303号创智天地科教广场3期11号楼7楼
邮编：200433
电话：(86.21) 6072-6200
传真：(86.21) 6082-1960

上海新思大厦办公室

地址：上海市漕河泾开发区宜山路926号新思大厦11层
邮编：200233
电话：(86.21) 6057-9100
传真：(86.21) 6083-5350

广州国际金融广场办公室

地址：广州市天河区珠江新城华夏路8号合景国际金融广场18楼
邮编：510623
电话：(86.20) 8513-2000
传真：(86.20) 8513-2380

成都中海国际中心办公室

地址：成都市高新区交子大道177号中海国际中心7楼B座02-06单元
邮编：610041
电话：(86.28) 8530-8600
传真：(86.28) 8530-8699

深圳飞亚达科技大厦办公室

地址：深圳市南山区高新南一道飞亚达科技大厦16层
邮编：518057
电话：(86.755) 8396-5000
传真：(86.591) 8601-3837

深圳德赛科技大厦办公室

地址：深圳市南山区高新南一道德赛科技大厦8层0801-0803单元
邮编：518057
电话：(86.755) 8660-7100
传真：(86.755) 2167-1299

大连办公室

地址：大连软件园东路23号大连软件园15号楼502
邮编：116023
电话：(86.411) 8465-6000
传真：(86.755) 8465-6499

苏州办公室

地址：苏州工业园区星湖街328号苏州国际科技园5期11幢1001室
邮编：215123
电话：(86.512) 8666-5000
传真：(86.512) 8187-7838

沈阳办公室

地址：沈阳市和平区青年大街390号皇朝万鑫国际大厦A座39层3901&3911室
邮编：110003
电话：(86.24) 8393-8700
传真：(86.24) 2353-0585

济南办公室

地址：济南市冻源大街150号中信广场11层1113单元
邮编：250011
电话：(86.531) 6861-1900
传真：(86.531) 8518-1133

南京办公室

地址：南京市玄武区洪武北路55号置地广场19层1911室
邮编：210018
电话：(86.25) 8579-7500
传真：(86.25) 8476-5226

西安办公室

地址：西安市高新区科技二路72号西安软件园零壹广场主楼1401室
邮编：710075
电话：(86.29) 8834-3400
传真：(86.25) 8833-9829

重庆办公室

地址：重庆市渝中区邹容路68号大都会商厦1611室
邮编：400010
电话：(86.23) 6037-5600
传真：(86.23) 6370-8700

杭州办公室

地址：杭州市西湖区杭大路15号嘉华国际商务中心810&811室
邮编：310007
电话：(86.571) 8168-3600
传真：(86.571) 8717-5299

福州办公室

地址：福州市五四路158号环球广场1601室
邮编：350003
电话：(86.591) 8621-5050
传真：(86.591) 8801-0330

青岛办公室

地址：青岛市香港中路76号颐中皇冠假日酒店909室
邮编：266071
电话：(86.532) 8571-8888
传真：(86.591) 8571-6666

南昌办公室

地址：江西省南昌市西湖区沿江中大道258号
皇冠商务广场10楼1009室
邮编：330025
电话：(86.791) 8612-1000
传真：(86.791) 8657-7693

呼和浩特办公室

地址：内蒙古自治区呼和浩特市新城区迎宾北路7号
大唐金座19层北侧1902-1904室
邮编：010051
电话：(86.471) 3941-600
传真：(86.471) 5100-535

郑州办公室

地址：河南省郑州市中原区中原中路220号
裕达国际贸易中心A座2015室
邮编：450007
电话：(86.371) 6755-9500
传真：(86.371) 6797-2085

武汉办公室

地址：武汉市江岸区中山大道1628号
武汉天地企业中心5号大厦23层2301单元
邮编：430010
电话：(86.27) 8221-2168
传真：(86.27) 8221-2168

长沙办公室

地址：长沙市芙蓉区韶山北路159号通程国际大酒店1311-1313室
邮编：410011
电话：(86.731) 8977-4100
传真：(86.731) 8425-9601

石家庄办公室

地址：石家庄市中山东路303号石家庄世贸广场酒店14层1402室
邮编：050011
电话：(86.311) 6670-8080
传真：(86.311) 8667-0618

昆明办公室

地址：昆明市三市街六号柏联广场写字楼11层1103A室
邮编：650021
电话：(86.871) 6402-4600
传真：(86.871) 6361-4946

合肥办公室

地址：安徽省合肥市蜀山区政务新区怀宁路1639号平安大厦18层1801室
邮编：230022
电话：(86.551) 6595-8200
传真：(86.551) 6371-3182

南宁办公室

地址：广西省南宁市青秀区民族大道136-2号华润大厦B座2302室
邮编：530028
电话：(86.771) 391-8400
传真：(86.771) 577-5500



Oracle Multitenant

2013 年 6 月

作者: Bryn Llewellyn

公司网址: <http://www.oracle.com> (英文)

中文网址: <http://www.oracle.com/cn> (简体中文)

销售中心: 800-810-0161

售后服务热线: 800-810-0366

培训服务热线: 800-810-9931

欢迎访问:

<http://www.oracle.com> (英文)

<http://www.oracle.com/cn> (简体中文)

版权© 2014 归 Oracle 公司所有。未经允许, 不得以任何形式和手段复制和使用。

本文的宗旨只是提供相关信息, 其内容如有变动, 恕不另行通知。Oracle 公司对本文内容的准确性不提供任何保证, 也不做任何口头或法律形式的其他保证或条件, 包括关于适销性或符合特定用途的所有默示保证和条件。本公司特别声明对本文档不承担任何义务, 而且本文档也不能构成任何直接或间接的合同责任。未经 Oracle 公司事先书面许可, 严禁将此文档为了任何目的, 以任何形式或手段(无论是电子的还是机械的)进行复制或传播。

Oracle 是 Oracle 公司和/或其分公司的注册商标。其他名字均可能是各相应公司的商标。